VIETNAM NATIONAL UNIVERSITY – HCMC

UNIVERSITY OF SCIENCE

Faculty of Information and Technology

Software Engineering Department

Software Design

Project name: Master Interview

Course: Introduction to Software Engineering

Class: 24C10

Students:

24127186 - Giáp Đăng Khoa

24127518 - Trần Ngọc Quân

24127136 - Nguyễn Thanh Trọng

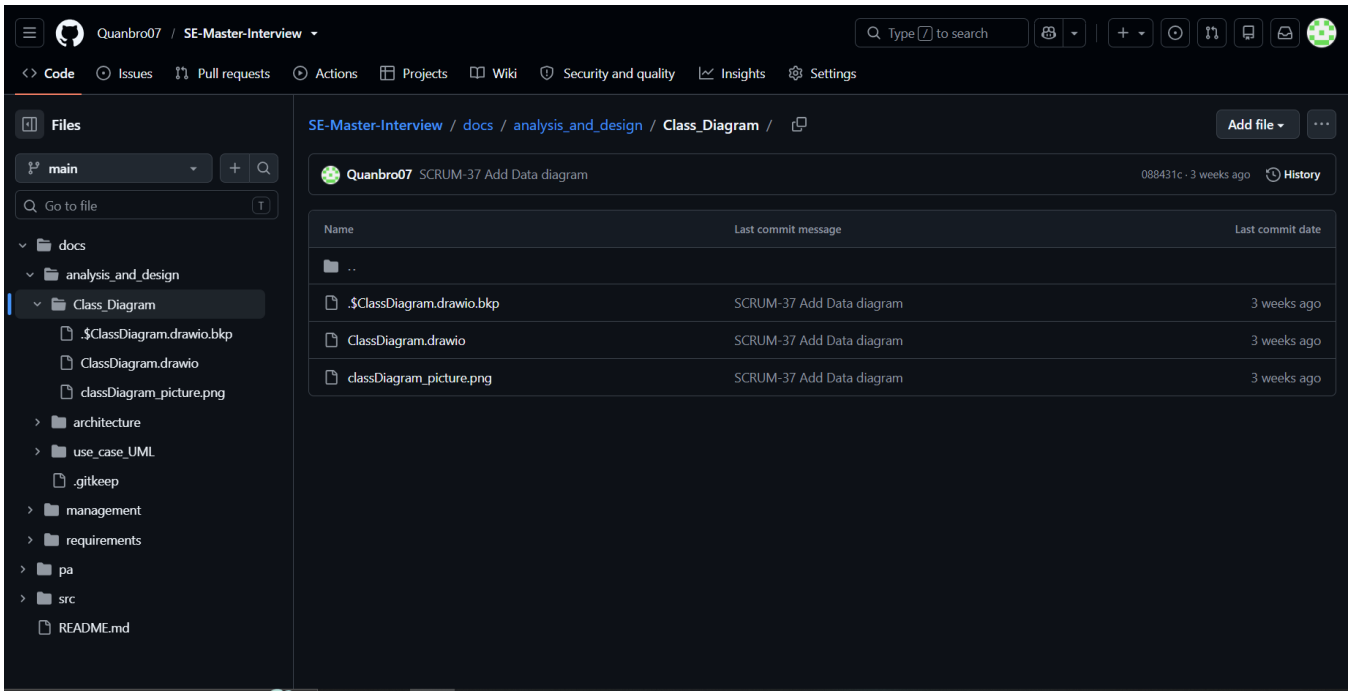
24127137 - Nguyễn Thanh Trúc

Table of Contents

1 Member Contribution Assessment

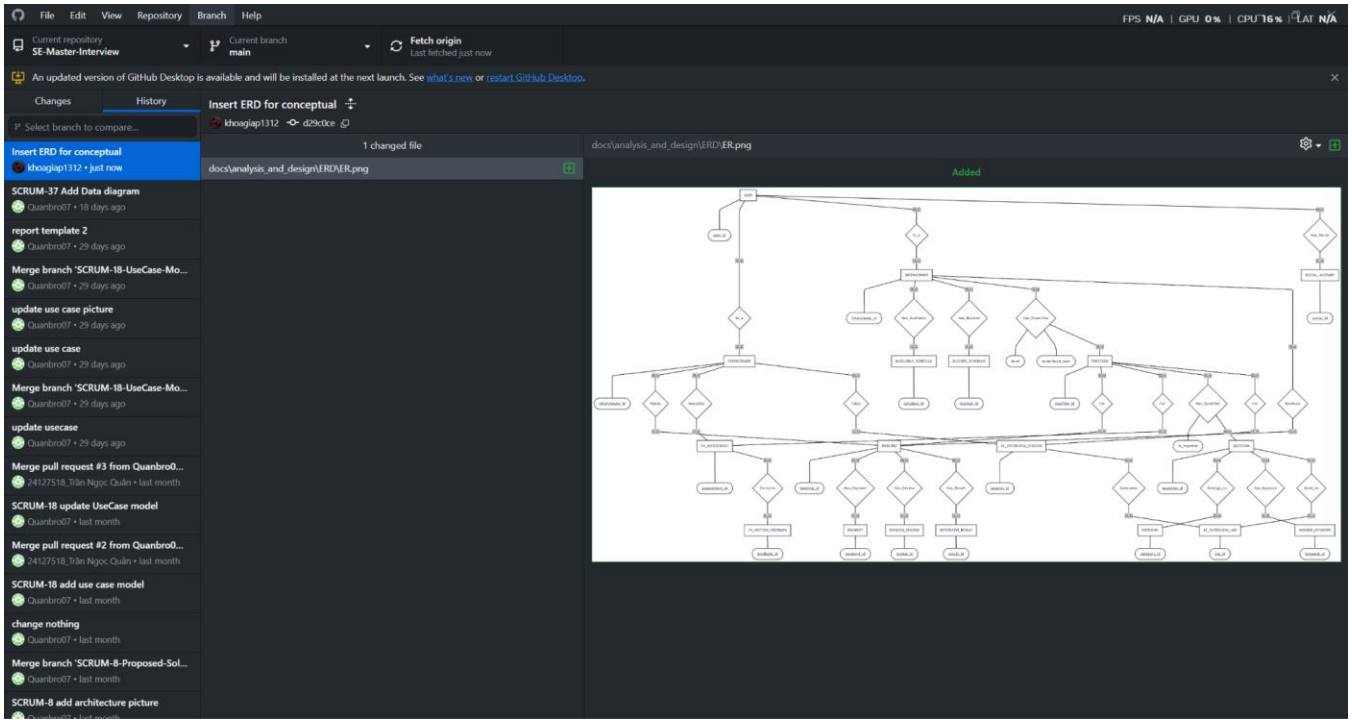
24127518 - Trần Ngọc Quân - Contribution (25%)

Task name	Summary
Design: Class Diagram	Illustrate the static structure of the system by defining the core classes, their attributes, operations (methods), and the relationships/associations between them.
Write Report: Class Specification	Document the Class Specification for Project.



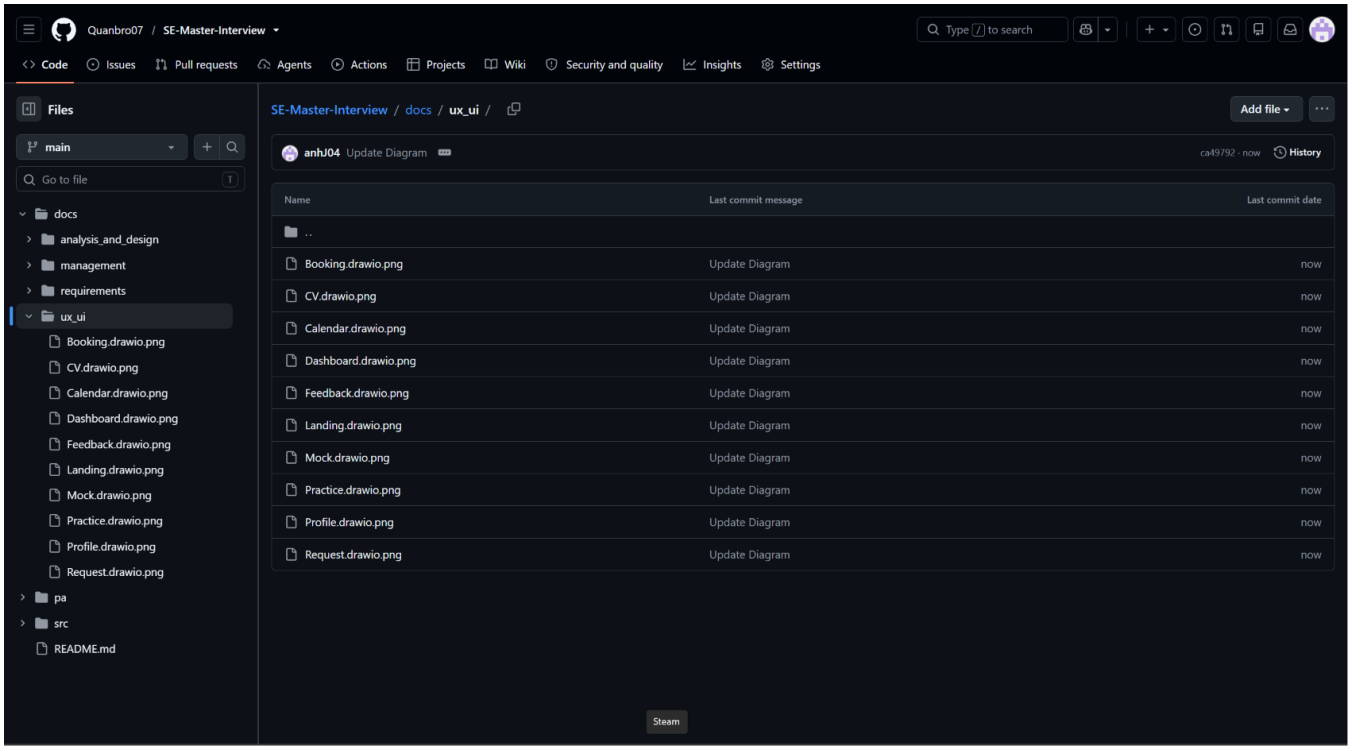
24127186 - Giáp Đăng Khoa- Contribution (25%)

Task name	Summary
Design: Concept Model	Create a high-level conceptual data model that identifies the key domain entities, their attributes, and semantic relationships to serve as a foundation for the database schema.
Write Report: Data Specification	Document the Data Specification



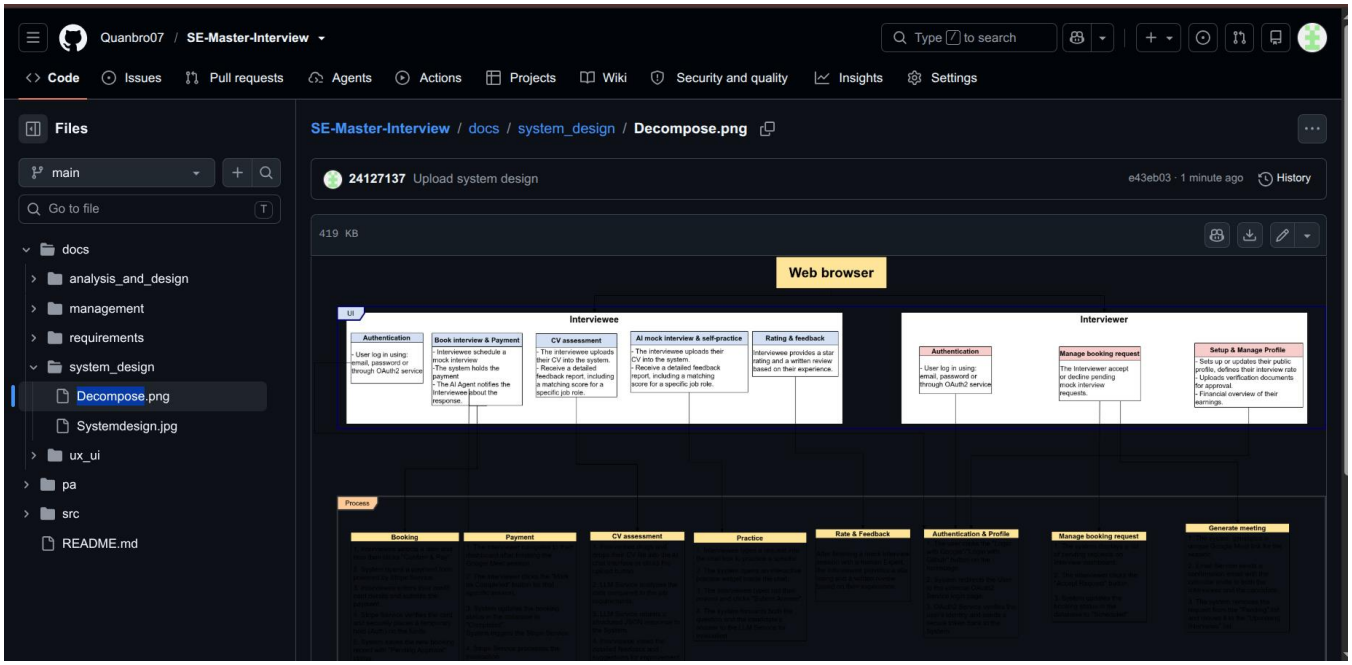
24127136 - Nguyễn Thanh Trọng - Contribution (25%)

Task name	Summary
Design: Screen Diagram	Design the UI wireframes and screen flows, visualizing the layout of user interfaces and how users will navigate between different pages of the application.
Write Report: Screen Specification	Document Screen Specification



24127137- Nguyễn Thanh Trúc - Contribution (25%)

Task name	Summary
Design: Architecture Diagram	Design the overall, high-level structural framework of the system, illustrating how major components, servers, databases, and external services interact with one another.
Design: System Decomposition Tree	Document Requirements


Quanbro07 / SE-Master-Interview
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
docs
analysis_and_design
management
requirements
system_design
Decompose.png
Systemdesign.jpg
ux_ui
pa
src
README.md
SE-Master-Interview / docs / system_design / Decompose.png
24127137 Upload system design
e43eb03 · 1 minute ago
History
419 KB
Web browser
UI
Interviewee
Authentication
- User log in using: email, password or through OAuth2 service
Book interview & Payment
- Interviewee schedule a mock interview
- The system holds the payment
- The AI Agent notifies the Interviewee about the response.
CV assessment
- The interviewee uploads their CV into the system.
- Receive a detailed feedback report, including a matching score for a specific job role.
AI mock interview & self-practice
- The interviewee uploads their CV into the system.
- Receive a detailed feedback report, including a matching score for a specific job role.
Rating & feedback
Interviewee provides a star rating and a written review based on their experience.
Interviewer
Authentication
- User log in using: email, password or through OAuth2 service
Manage booking request
The Interviewer accept or decline pending mock interview requests.
Setup & Manage Profile
- Sets up or updates their public profile, defines their interview rate
- Uploads verification documents for approval.
- Financial overview of their earnings.
Process
Booking
Payment
CV assessment
Practice
Rate & Feedback
Authentication & Profile
Manage booking request
Generate meeting

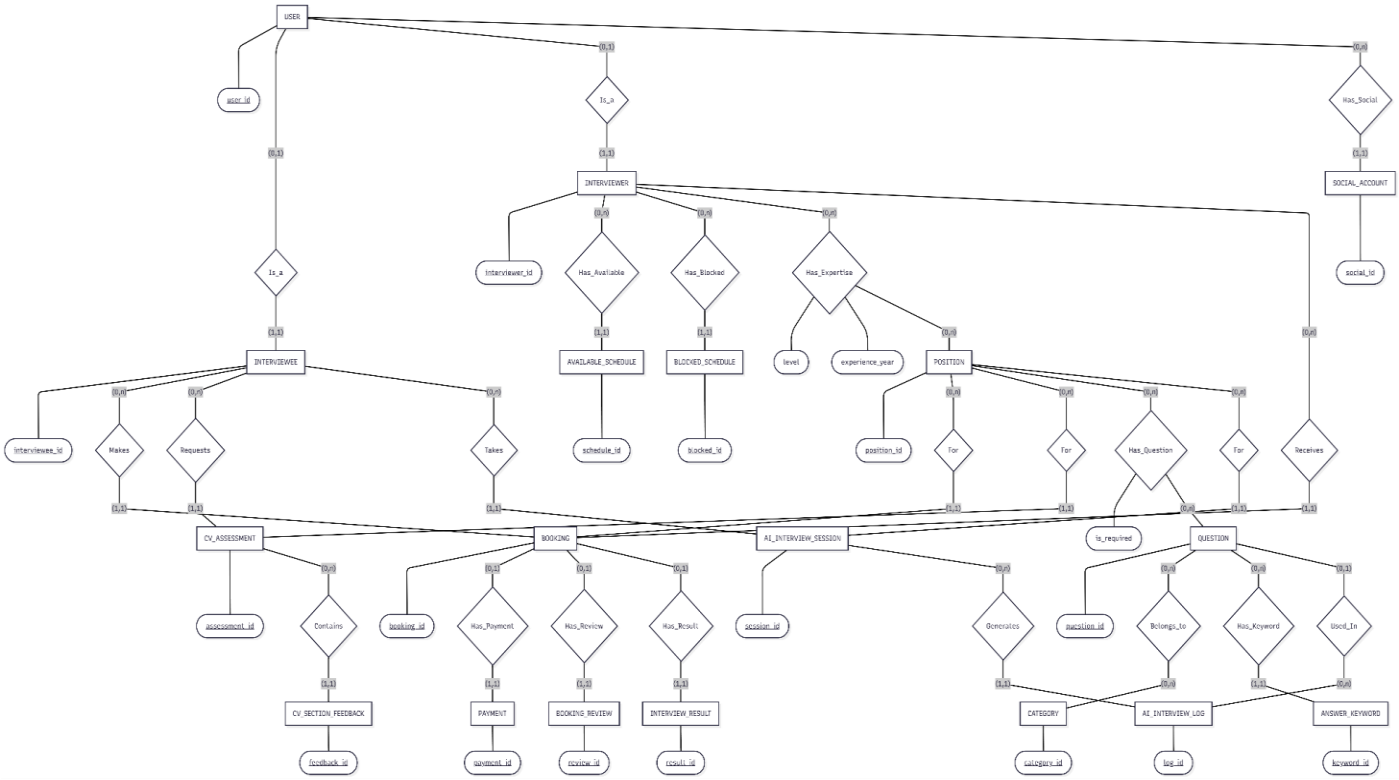
2 Conceptual Model

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127186

Edited by:

Reviewed by:



3 Architectural Design

1.1 Architecture Diagram

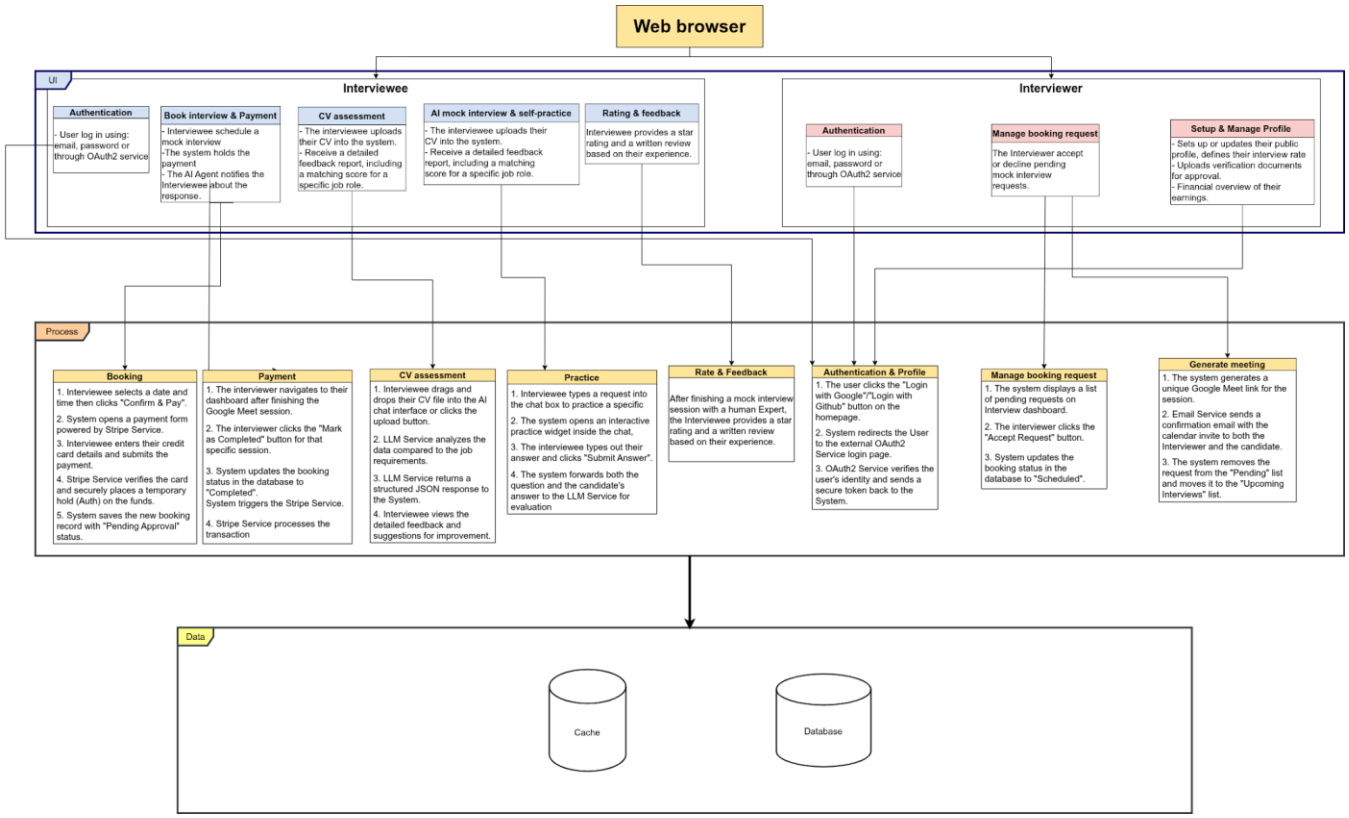
Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: Trúc

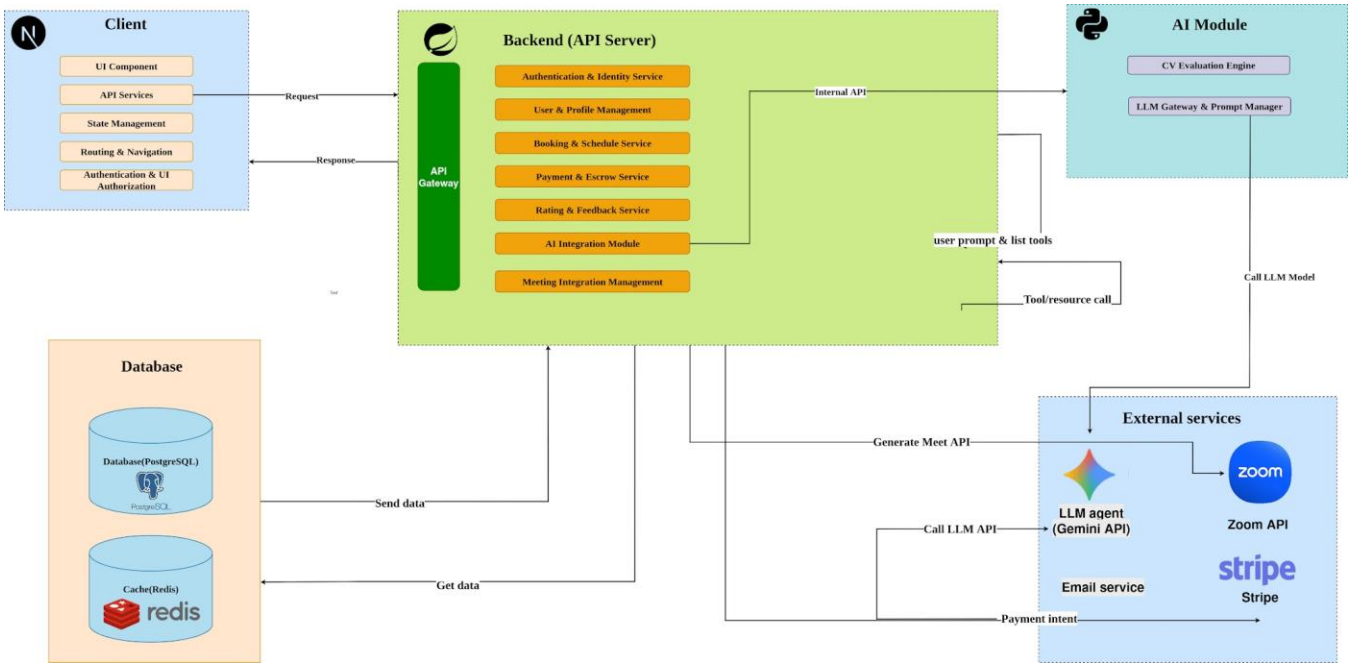
Edited by:

Reviewed by:

[Present a **system decomposition tree diagram**, indicating the components of the system]



[Present an overall system architecture diagram, illustrating the relationships between the **MAIN components** identified in the system decomposition tree]



1. **Architecture:**

The system is designed with a **3-Tier architecture** combined with a **client-server** model, including:

- **UI Tier (Client):** A Next.js framework responsible for the user interface, state management, and routing & navigation.
- **Process Tier (Backend & AI Module):** Handles core business logic, divided into independent service and an isolated AI module.
- **Data Tier (Database):** Use PostgreSQL for persistent relational data and Redis for caching to optimize read performance and data retrieval.

2. **Design Patterns:**

- **API Gateway Pattern:** The backend utilizes an API gateway as the single entry point to receive, resolve, and route all incoming frontend requests to the appropriate internal modules.
- **Service-Oriented:** System functionalities are domain-driven, divided into User Management, Payment Service, and Rating Service,etc.
- Use **Adapter Pattern** to communicate with third-party services, isolating the core system logic from external API updates or changes.

3. **Plug-in Mechanism**

- **MCP (Model Context Protocol):** The system integrates an MCP Client. This acts as a bridge that allows the LLM to interact back with the internal system.
- **Independent AI Module:** CV Evaluation are separated into a standalone module to prevent bottlenecking standard synchronous business APIs.

1.2 Class Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: Quân

Edited by:

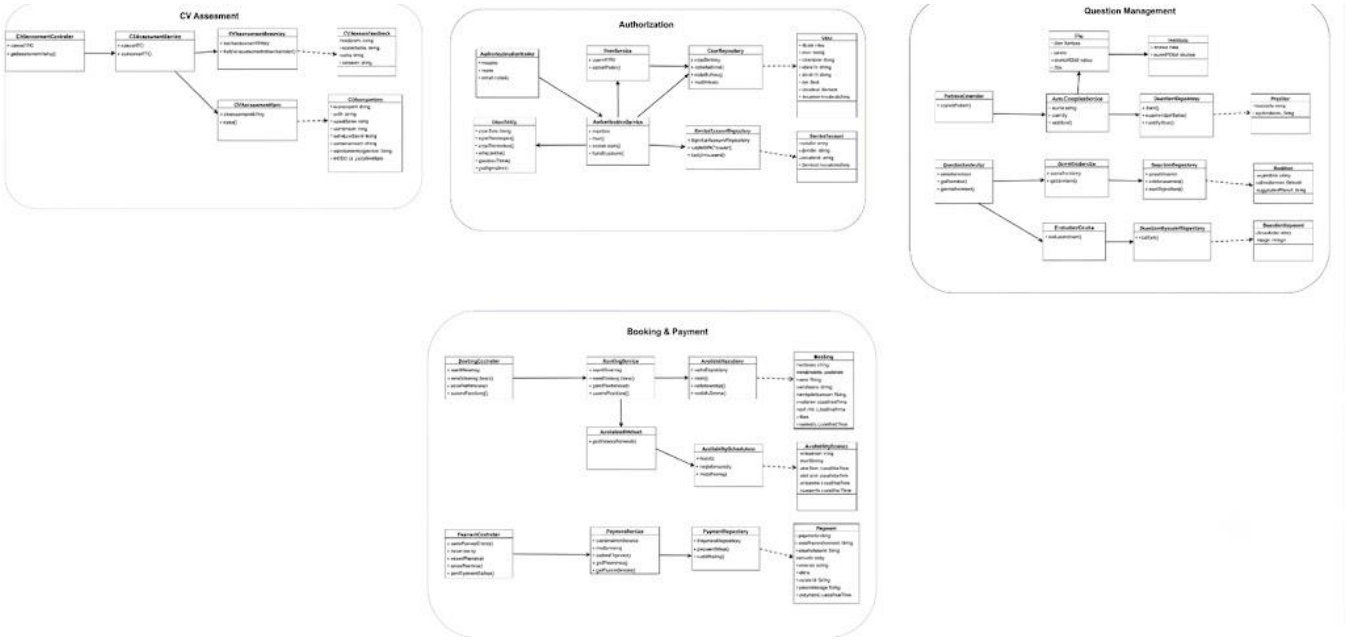
Reviewed by:

This diagram illustrates the overall system architecture based on the Spring Boot layered pattern (Controller, Service, Repository, and Entity). Given the large number of classes, this overarching diagram provides a high-level view of component dependencies.

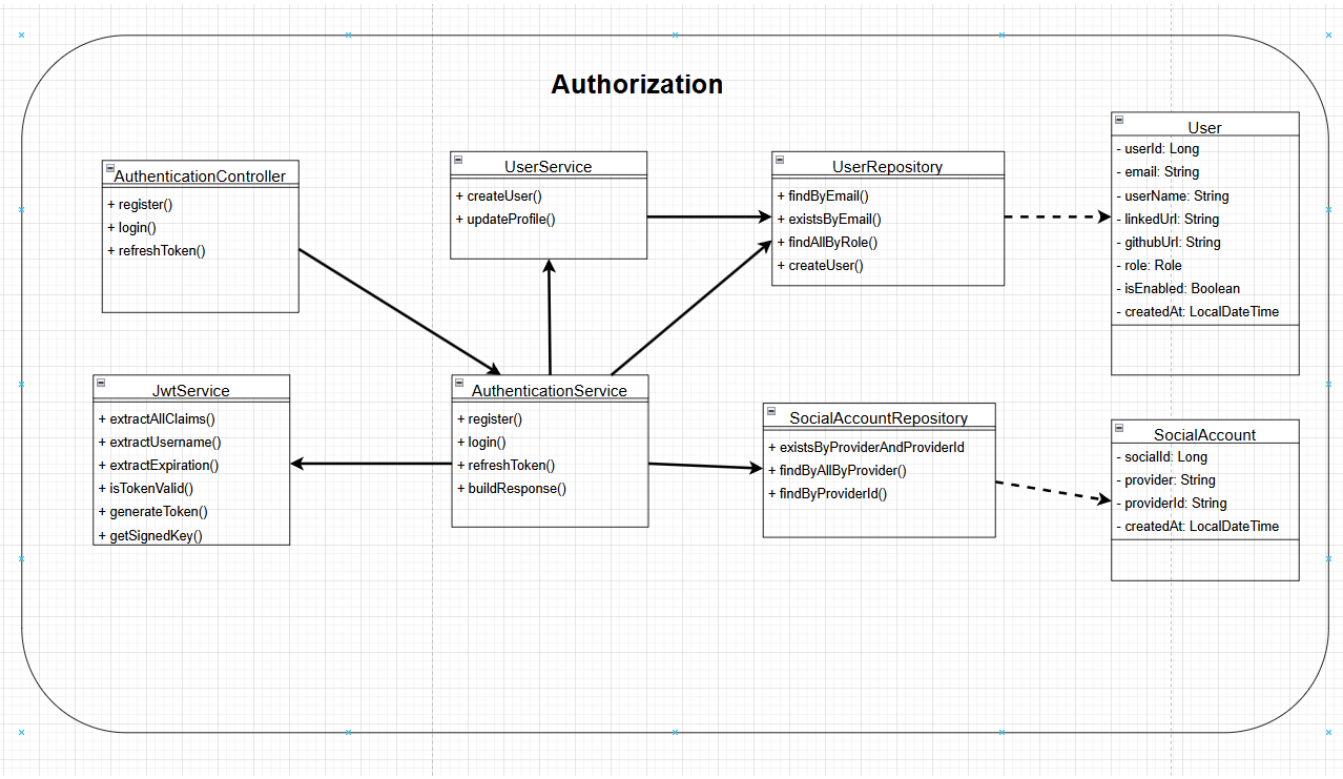
To clearly show class names, relationship types, and cardinalities without overcrowding the visual space, the system is divided into specific business modules.

Design Note:

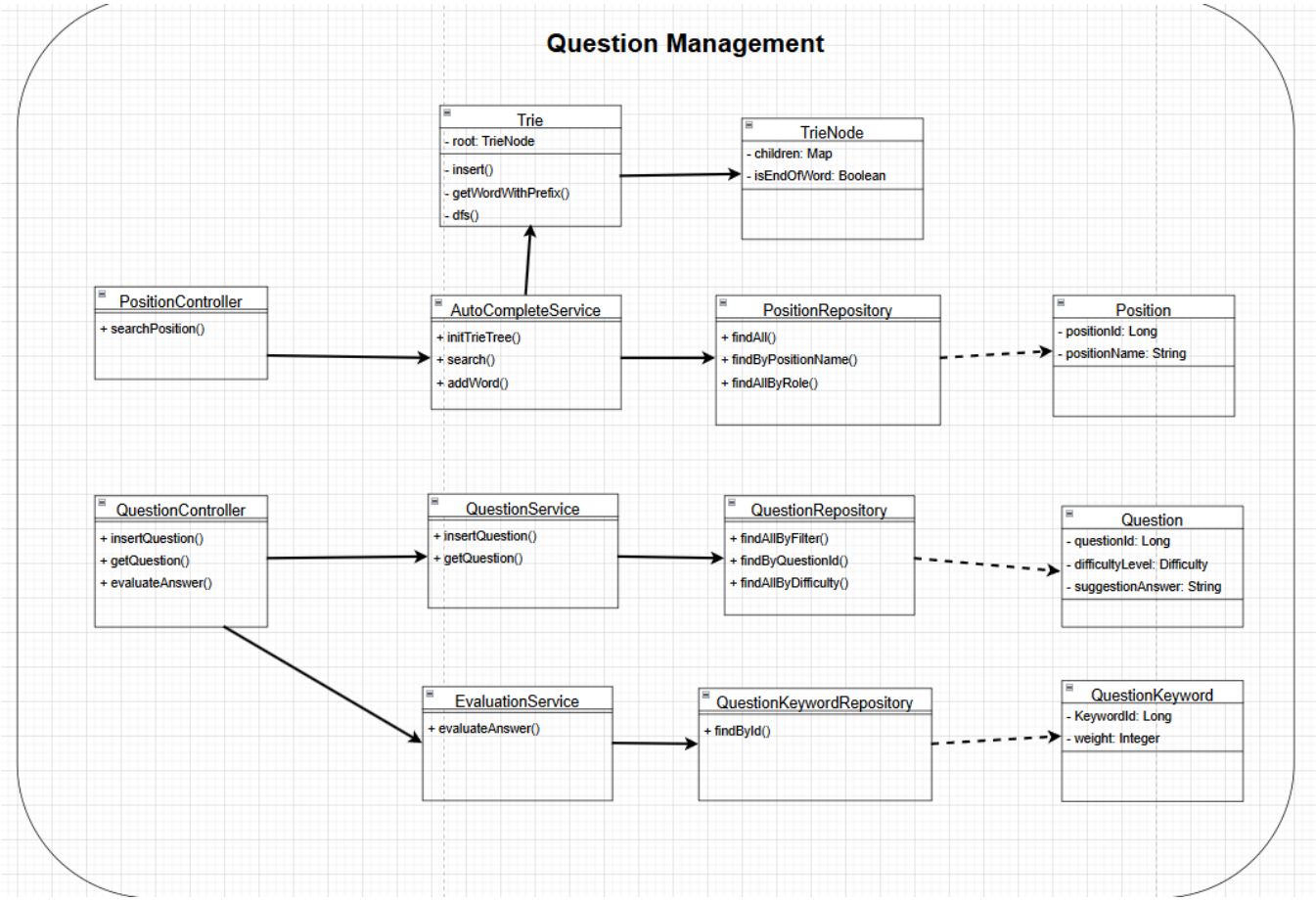
- **Controllers, Services, and Repositories:** Core methods are explicitly listed.
- **Entities:** To optimize space and avoid redundancy, Entity classes are simplified to show only their names and data flow dependencies. Full details regarding their attributes, data types, and complex relationships are comprehensively documented in the **Data Diagram (ERD)** section of this report.



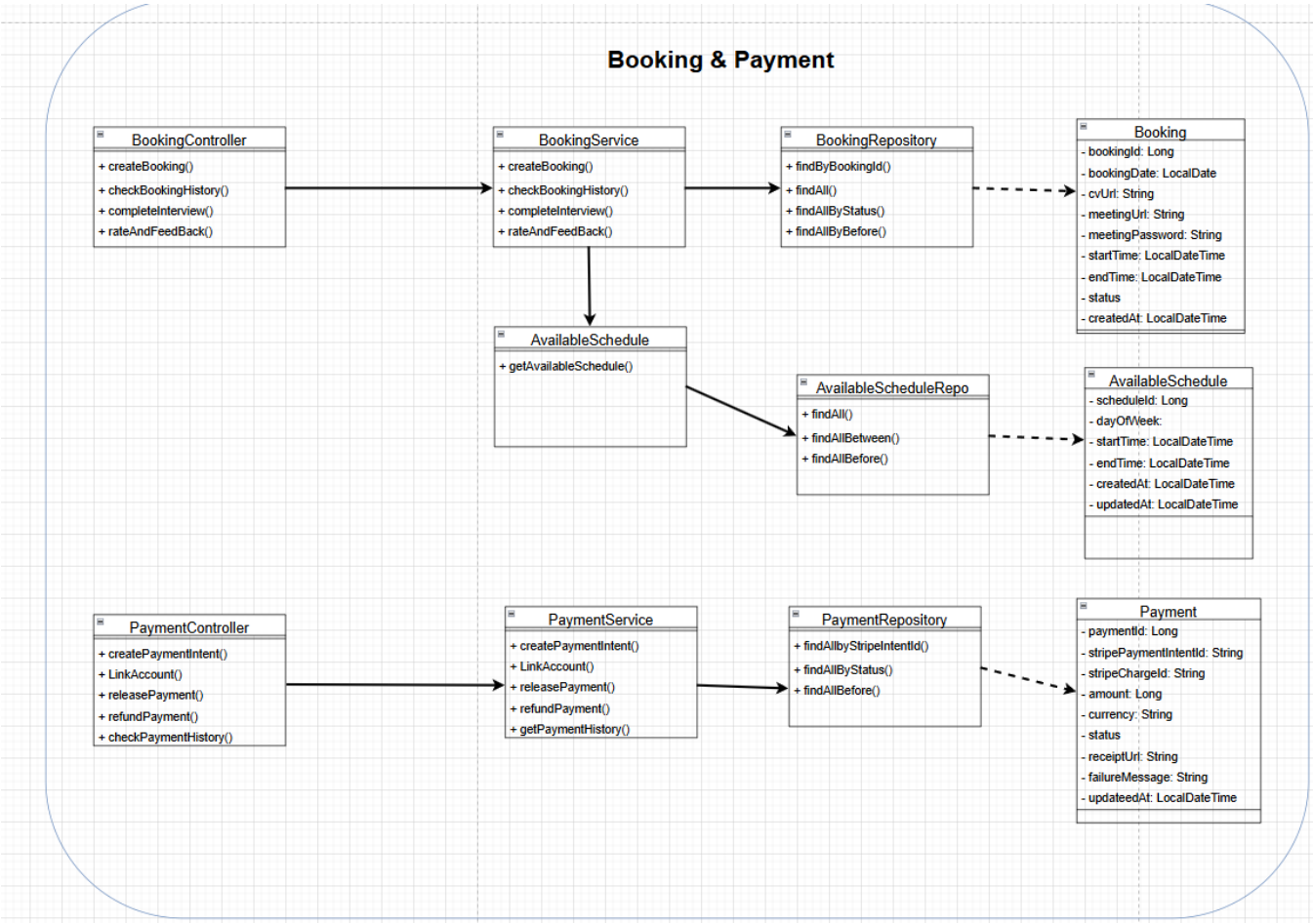
[Overall Class Diagram]



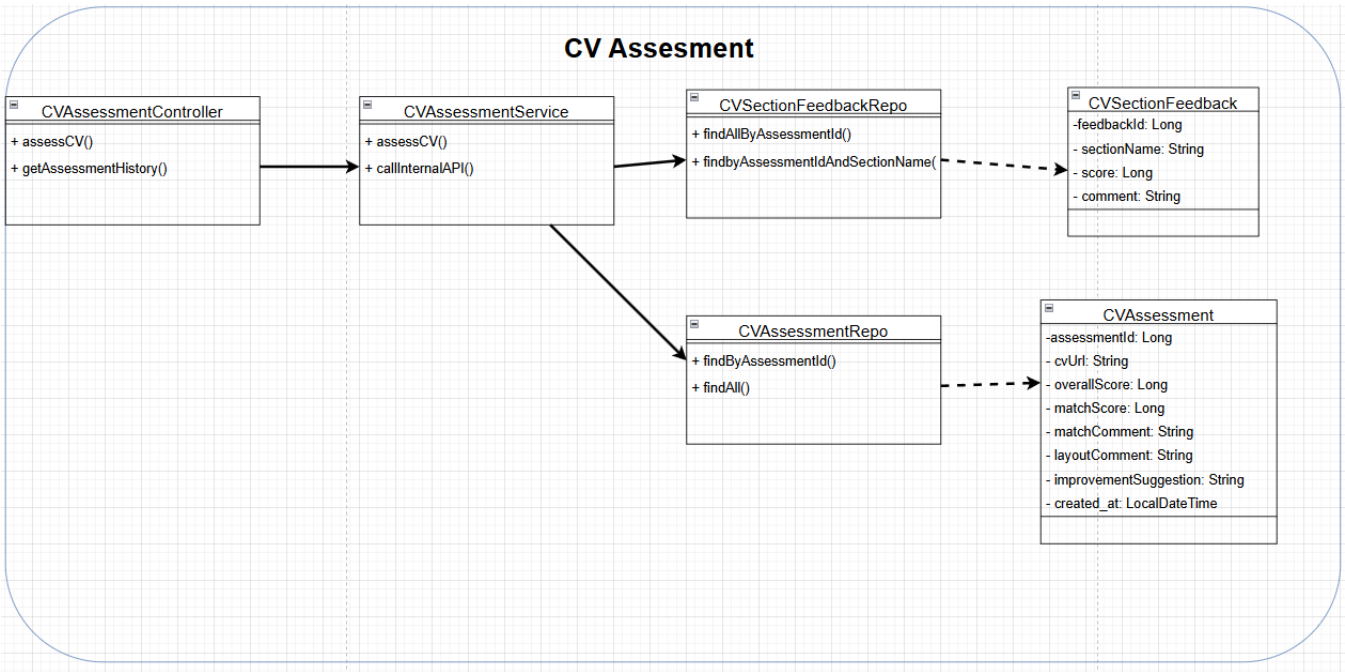
[Authorization]



[Question Management]



[Booking & Payment]



[CV Assessment]

1.3 Class Specifications

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by:

Edited by:

Reviewed by:

[Students should select and present the specifications of a few (9-10) of the most important classes .]

1.3.1 Class AuthenticationService

Inherits from: None

Description: Handles core business logic for user authorization, including user registration, login authentication, and JWT token management.

Seq	Property	Modifier	Constraint	Description
1	userRepository	public	None	Dependency injection from User data manipulation
2	jwtService		None	Dependency injection for token operations.

Seq	Operation	Modifier	Constraint	Description
1	register()	public	Valid payload	Create a new user account in the system
2	login()	public	Valid credentials	Authenticates user and returns JWT token
3	refreshToken()	public	Valid token	Generate a new access token using a refresh token
4	buildResponse()	public	None	Formats the standard authentication response

1.3.2 Class User (Entity)

Inherits from: None

Description: Represents a user entity in the database, containing authentication and profile details.

Seq	Property	Modifier	Constraint	Description
1	userId	private	Primary Key	Unique identifier for the user.
2	email	private	Unique, Not Null	User's email address for login.
3	userName	private	None	Display name of the user.
4	role	private	Not Null	Enum defining user privileges.
5	isEnabled	private	Default true	Flag indicating if the account is active.

Seq	Operation	Modifier	Constraint	Description
1	Getters & Setters	public	None	Standard accessors and mutators for all properties.

1.3.3 Class AutoCompleteService

Inherits from: None

Description: Provides fast algorithmic search and autocomplete suggestions for question keywords using a Trie data structure.

Seq	Property	Modifier	Constraint	Description
1	trie	private	None	Instance of the Trie data structure.

Seq	Operation	Modifier	Constraint	Description
1	initTrieTree()	public	None	Initializes and populates the Trie from the database.

Seq	Operation	Modifier	Constraint	Description
2	search()	public	Prefix != null	Retrieves a list of suggested words based on a prefix.
3	addWord()	public	Word != null	Dynamically inserts a new keyword into the Trie.

1.3.4 Class Trie

Inherits from: None

Description: Provides fast algorithmic search and autocomplete suggestions for question keywords using a Trie data structure.

Seq	Property	Modifier	Constraint	Description
1	root	private	Not Null	The root node of the Trie.

Seq	Operation	Modifier	Constraint	Description
1	insert()	public	String input	Inserts a string character by character into nodes.
2	getWordWithPrefix()	public	String Prefix	Finds the node matching the prefix prefix
3	dfs	private	None	Depth-First Search helper to traverse and collect words.

1.3.5 Class BookingService

Inherits from: None

Description: Manages the business flow of scheduling, validating, and finalizing interview bookings.

Seq	Property	Modifier	Constraint	Description
1	bookingRepository	private	None	Dependency for Booking persistence.

Seq	Operation	Modifier	Constraint	Description
1	createBooking()	public	Valid schedule	Reserves a time slot and generates meeting details.
2	checkBookingHistory()	public	Valid user ID	Retrieves a list of past and upcoming bookings
3	completeInterview()	public	Valid booking ID	Updates booking status to complete.
4	rateAndFeedback()	public	Valid booking ID	Submits post-interview evaluation scores.

1.3.6 Class Booking

Inherits from: None

Description: Stores comprehensive details about a scheduled interview meeting.

Seq	Property	Modifier	Constraint	Description
1	bookingRepository	private	Primary Key	Unique identifier for the booking.
2	meetingUrl	private	URL format	The generated video conferencing link.
3	bookingDate	private	None	The Booking date
4	cvUrl	private	None	CV uses for interview
5	startTime	private	> Current Time	The scheduled start date and time.
6	endTime	private	> startTime	The end time of interview
7	status	private	Enum	Current state (e.g., PENDING, COMPLETED).

Seq	Operation	Modifier	Constraint	Description
1	Getters & Setters	public	None	Standard accessors and mutators.

1.3.7 Class AIInterviewService

Inherits from: None

Description: Orchestrates the flow of mock interviews, interacting with internal AI models to evaluate user responses.

Seq	Property	Modifier	Constraint	Description
1	interviewSessionRepo	private	None	Dependency for session persistence.

Seq	Operation	Modifier	Constraint	Description
1	createInterview()	public	None	Initializes a new AI mock interview session.
2	callInternalApi()	private	Network active	Communicates with the Python AI service for analysis.
3	completeInterview()	public	Valid session ID	Finalizes the session and calculates overall score

1.3.8 Class CVAssessmentService

Inherits from: None

Description: Handles the logic for parsing, submitting, and retrieving AI evaluations of user resumes (CVs).

Seq	Property	Modifier	Constraint	Description
1	cvAssessmentRepo	private	None	Dependency for CV assessment persistence.

Seq	Operation	Modifier	Constraint	Description
1	assessCV()	public	Valid File URL	Triggers the CV scanning and scoring process.
2	callInternalApi()	private	Network active	Sends CV data to the AI scoring engine.

1.3.9 Class JwtService

Inherits from: None

Description: Utility service responsible for generating, signing, and parsing JSON Web Tokens.

Seq	Property	Modifier	Constraint	Description
1	secretKey	private	Secure String	Secret key used for signing JWTs.

Seq	Operation	Modifier	Constraint	Description
1	generateToken()	public	Valid UserDetails	Creates an encoded JWT for a specific user
2	extractAllClaims()	public	Valid Token	Decodes the token to claims.
3	extractExpiration()	public	Valid Token	Decodes the token to retrieve the expiration.
4	extractUsername()	public	Valid Token	Decodes the token to retrieve the username.
5	isTokenValid()	public	Valid Token	Verifies signature and checks expiration date.

4 Data Design

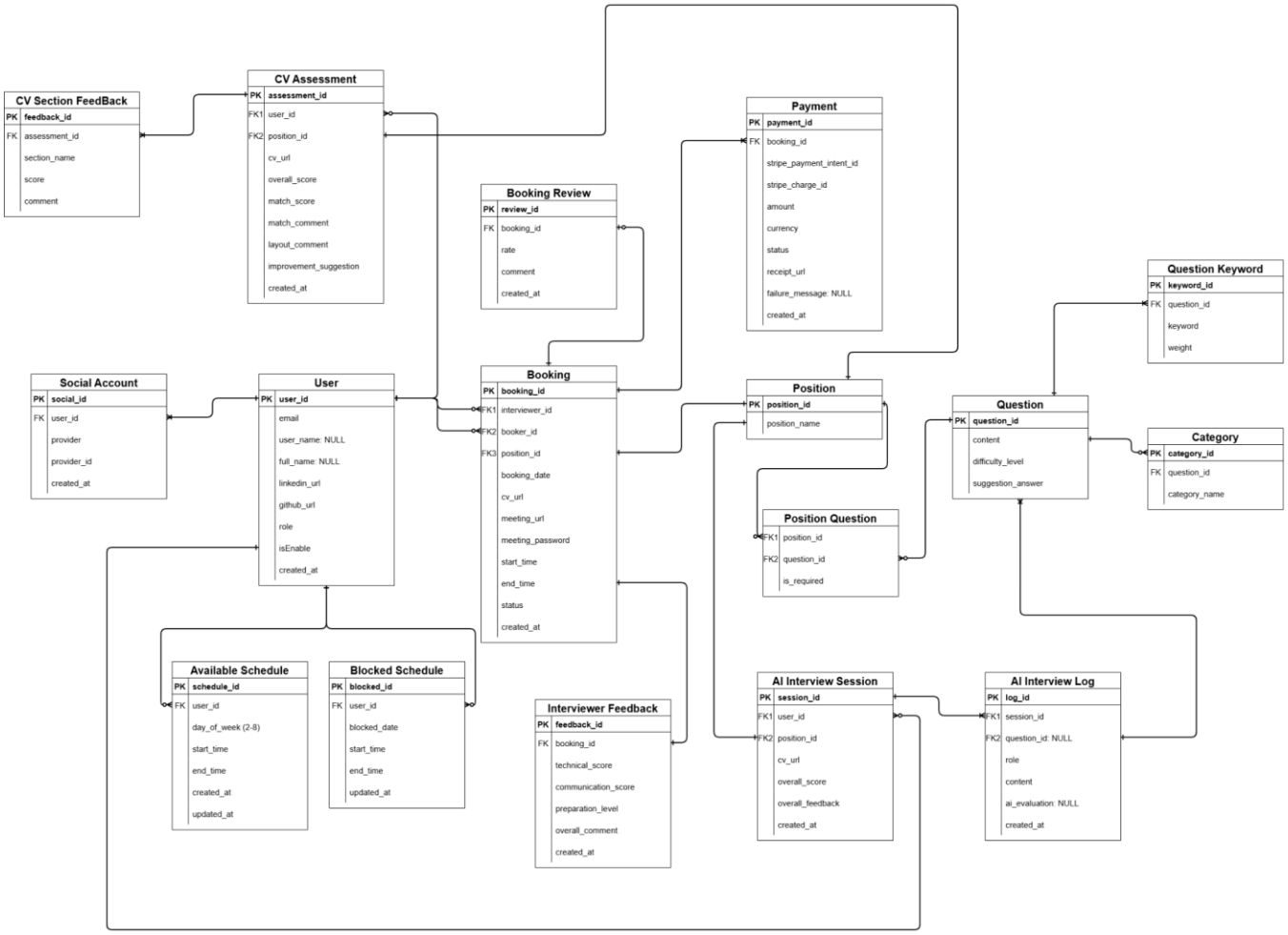
1.4 Data Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127518 - Trần Ngọc Quân

Edited by:

Reviewed by:



[Draw the data diagram of the system, identifying the data components that need to be stored and illustrating the relationships between them]

1.5 Data Specification

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127186

Edited by:

Reviewed by: 24127518

1. Table "user"

Stores account information of system users (interviewees, interviewers, administrators).

Attribute	Data Type	Constraint	Description
user_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the user
email	VARCHAR(255)	NOT NULL, UNIQUE	User's login email address
user_name	VARCHAR(100)	NULLABLE	Display name / username of the user
full_name	VARCHAR(255)	NULLABLE	Full name of the user
linkedin_url	VARCHAR(500)	NULLABLE	Link to the user's LinkedIn profile
github_url	VARCHAR(500)	NULLABLE	Link to the user's GitHub profile
role	VARCHAR(20)	NOT NULL, ENUM('INTERVIEWEE','INTERVIEWER','ADMIN')	Role of the user in the system
isEnable	BOOLEAN	NOT NULL, DEFAULT TRUE	Account activation status
created_at	DATETIME	NOT NULL	Timestamp when the account was created

2. Table "social_account"

Stores social login (OAuth2) links associated with a user.

Attribute	Data Type	Constraint	Description
social_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the social account link
user_id	BIGINT	FK -> user.user_id, NOT NULL	User who owns this link
provider	VARCHAR(50)	NOT NULL	Login provider (Google, GitHub, etc.)
provider_id	VARCHAR(255)	NOT NULL, UNIQUE (with provider)	User identifier on the provider's side
created_at	DATETIME	NOT NULL	Timestamp when the account was linked

3. Table "interviewee"

Extends the user table when a user has the interviewee role (1-1 inheritance from user).

Attribute	Data Type	Constraint	Description
interviewee_id	BIGINT	PK, FK -> user.user_id	Interviewee identifier (shares the same id as user)
subscription_expired_date	DATE	NULLABLE	Expiration date of the current subscription plan
updated_at	DATETIME	NOT NULL	Timestamp of the last update

4. Table "interviewer"

Extends the user table when a user has the interviewer role (1-1 inheritance from user).

Attribute	Data Type	Constraint	Description

interviewer_id	BIGINT	PK, FK -> user.user_id	Interviewer identifier (shares the same id as user)
isStripeConnected	BOOLEAN	NOT NULL, DEFAULT FALSE	Whether the Stripe account has been connected to receive payments
stripe_id	VARCHAR(255)	NULLABLE	Stripe Connect account identifier
updated_at	DATETIME	NOT NULL	Timestamp of the last update

5. Table "interviewer_expertise"

Many-to-many junction table representing an interviewer's expertise for each position. Composite primary key: (interviewer_id, position_id).

Attribute	Data Type	Constraint	Description
interviewer_id	BIGINT	PK (composite), FK -> interviewer.interviewer_id	Interviewer who owns this expertise
position_id	BIGINT	PK (composite), FK -> position.position_id	Position of expertise
level	VARCHAR(50)	NOT NULL, ENUM('JUNIOR','MIDDLE','SENIOR', ...)	Expertise level of the interviewer for this position
experience_year	INT	NOT NULL, CHECK(experience_year >= 0)	Number of years of experience in this position

6. Table "position"

Catalog of job/interview positions (e.g., Backend Developer, Data Analyst, etc.).

Attribute	Data Type	Constraint	Description
position_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the position
position_name	VARCHAR(255)	NOT NULL, UNIQUE	Name of the position

7. Table "cv_assessment"

Stores CV assessment results of an interviewee for a specific position.

Attribute	Data Type	Constraint	Description
assessment_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the CV assessment
interviewee_id	BIGINT	FK->interviewee.interviewee_id, NOT NULL	Interviewee being assessed
position_id	BIGINT	FK->position.position_id, NOT NULL	Position used as reference for the CV assessment
cv_url	VARCHAR(500)	NOT NULL	Link to the uploaded CV file
overall_score	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Overall score of the CV
match_score	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Score indicating fit with the position
match_comment	TEXT	NULLABLE	Comment on position fit
layout_comment	TEXT	NULLABLE	Comment on CV layout/formatting
improvement_suggestion	TEXT	NULLABLE	Suggestions to improve the CV

created_at	DATETIME	NOT NULL	Timestamp when the assessment was created

8. Table "cv_section_feedback"

Detailed feedback for each section of the CV, belonging to a cv_assessment.

Attribute	Data Type	Constraint	Description
feedback_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the section feedback
assessment_id	BIGINT	FK->cv_assessment.assessment_id, NOT NULL	CV assessment this feedback belongs to
section_name	VARCHAR(100)	NOT NULL	Name of the CV section (e.g.,Education, Experience, Skills)
score	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Score of this section
comment	TEXT	NULLABLE	Detailed comment for this section

9. Table "available_schedule"

Interviewer's recurring weekly availability time slots.

Attribute	Data Type	Constraint	Description
schedule_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the availability slot.
interviewer_id	BIGINT	FK->interviewer.interviewer_id, NOT NULL	Interviewer who owns this schedule.

day_of_week	TINYINT	NOT NULL, CHECK (2-8)	Day of week(2-8 ⇔ Monday-Sunday)
start_time	TIME	NOT NULL	Start time of availability.
end_time	TIME	NOT NULL, CHECK (end_time > start_time)	End time of availability.
created_at	DATETIME	NOT NULL	Timestamp when the record was created.
updated_at	DATETIME	NOT NULL	Timestamp of the last update.

10. Table "blocked_schedule"

Interviewer's one-off unavailability override for a specific date.

Attribute	Data Type	Constraint	Description
blocked_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the blocked slot
interviewer_id	BIGINT	FK -> interviewer.interviewer_id, NOT NULL	Interviewer whose schedule is blocked
blocked_date	DATE	NOT NULL	Specific date that is blocked
start_time	TIME	NOT NULL	Start time of the blocked period
end_time	TIME	NOT NULL, CHECK (end_time > start_time)	End time of the blocked period
updated_at	DATETIME	NOT NULL	Timestamp of the last update

11. Table "booking"

An interview booking between an interviewee and an interviewer for a specific position.

Attribute	Data Type	Constraint	Description
booking_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the booking
interviewer_id	BIGINT	FK -> interviewer.interviewer_id, NOT NULL	Interviewer being booked
interviewee_id	BIGINT	FK -> interviewee.interviewee_id, NOT NULL	Interviewee who made the booking
position_id	BIGINT	FK -> position.position_id, NOT NULL	Position of the interview
booking_date	DATE	NOT NULL	Date on which the interview takes place
cv_url	VARCHAR(500)	NULLABLE	CV attached to the interview
meeting_url	VARCHAR(500)	NULLABLE	Link to the online meeting room
meeting_password	VARCHAR(100)	NULLABLE	Meeting room

			password (if any)
start_time	DATETIME	NOT NULL	Interview start time
end_time	DATETIME	NOT NULL, CHECK (end_time > start_time)	Interview end time
status	VARCHAR(20)	NOT NULL, ENUM('PENDING','CONFIRMED','COMPLETED','CANCELLED')	Status of the booking
created_at	DATETIME	NOT NULL	Timestamp when the booking was created

12. Table "booking_review"

Interviewee's review of the interview after it has been completed.

Attribute	Data Type	Constraint	Description
review_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the review
booking_id	BIGINT	FK -> booking.booking_id, NOT NULL, UNIQUE	Booking being reviewed
rate	TINYINT	NOT NULL, CHECK (1-5)	Star rating
comment	TEXT	NULLABLE	Interviewee's comment
created_at	DATETIME	NOT NULL	Timestamp when the review was created

13. Table "interview_result"

Grading result of the interview, provided by the interviewer.

Attribute	Data Type	Constraint	Description
result_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the interview result
booking_id	BIGINT	FK -> booking.booking_id, NOT NULL, UNIQUE	Corresponding booking
technical_score	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Technical score
communication_score	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Communication skill score
preparation_level	DECIMAL(5,2)	NOT NULL, CHECK (0-100)	Interviewee's level of preparation
overall_comment	TEXT	NULLABLE	Overall comment
created_at	DATETIME	NOT NULL	Timestamp when the result was created

14. Table "payment"

Payment transaction associated with a booking (via Stripe).

Attribute	Data Type	Constraint	Description
payment_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the transaction
booking_id	BIGINT	FK -> booking.booking_id, NOT NULL	Booking being paid for

stripe_payment_intent_id	VARCHAR(255)	NOT NULL, UNIQUE	Stripe Payment Intent identifier
stripe_charge_id	VARCHAR(255)	NULLABLE	Stripe charge identifier
amount	DECIMAL(12, 2)	NOT NULL, CHECK (amount > 0)	Transaction amount
currency	VARCHAR(10)	NOT NULL, DEFAULT 'VND'	Currency unit
status	VARCHAR(20)	NOT NULL, ENUM('PENDING','SUCCEEDED','FAILED','REFUNDED')	Status of the transaction
receipt_url	VARCHAR(500)	NULLABLE	Link to the electronic receipt
failure_message	VARCHAR(500)	NULLABLE	Error message if the transaction failed
created_at	DATETIME	NOT NULL	Timestamp when the transaction was created

15. Table "category"

Catalog of interview question categories (e.g., Algorithms, Databases, Soft skills, etc.).

Attribute	Data Type	Constraint	Description
category_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the category
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the category

16. Table "question"

Bank of interview questions.

Attribute	Data Type	Constraint	Description
question_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the question
content	TEXT	NOT NULL	Content of the question
difficulty_level	VARCHAR(20)	NOT NULL, ENUM('EASY','MEDIUM','HARD')	Difficulty level of the question
suggestion_answer	TEXT	NULLABLE	Suggested reference answer

17. Table "question_category"

Many-to-many junction table between question and category. Composite primary key: (question_id, category_id).

Attribute	Data Type	Constraint	Description

question_id	BIGINT	PK (composite), FK -> question.question_id	The question
category_id	BIGINT	PK (composite), FK -> category.category_id	The category

18. Table "answer_keyword"

Keywords used to automatically score an answer to a question.

Attribute	Data Type	Constraint	Description
keyword_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the keyword
question_id	BIGINT	FK -> question.question_id, NOT NULL	Related question
keyword	VARCHAR(255)	NOT NULL	Keyword content
weight	INT	NOT NULL, CHECK (weight >= 0)	Weight of the keyword when scoring

19. Table "position_question"

Many-to-many junction table between position and question. Composite primary key: (position_id, question_id).

Attribute	Data Type	Constraint	Description
position_id	BIGINT	PK (composite), FK -> position.position_id	The position
question_id	BIGINT	PK (composite), FK -> question.question_id	The question
is_required	BOOLEAN	NOT NULL, DEFAULT FALSE	Whether the question is required for this position

20. Table "ai_interview_session"

AI mock interview session of an interviewee for a specific position.

Attribute	Data Type	Constraint	Description
session_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the AI interview session
interviewee_id	BIGINT	FK -> interviewee.interviewee_id, NOT NULL	Participating interviewee
position_id	BIGINT	FK -> position.position_id, NOT NULL	Position of the mock interview
cv_url	VARCHAR(500)	NULLABLE	CV used for the session
overall_score	DECIMAL(5,2)	NULLABLE, CHECK (0-100)	Overall score given by the AI
overall_feedback	TEXT	NULLABLE	Overall feedback from the AI
created_at	DATETIME	NOT NULL	Timestamp when the session was created

21. Table "ai_interview_log"

Conversation log (questions/answers) within an AI interview session.

Attribute	Data Type	Constraint	Description
log_id	BIGINT	PK, AUTO_INCREMENT	Unique identifier of the log entry

session_id	BIGINT	FK -> ai_interview_session.session_id, NOT NULL	Related interview session
question_id	BIGINT	FK -> question.question_id, NULLABLE	Question used (NULL if AI-generated freely)
role	VARCHAR(20)	NOT NULL, ENUM('AI','USER')	Role that generated the content
content	TEXT	NOT NULL	Content of the question/answer
ai_evaluation	TEXT	NULLABLE	AI's evaluation of the answer (if role = USER)
created_at	DATETIME	NOT NULL	Timestamp when the log entry was recorded

5 User Interface and User Experience Design

1.6 Screen Diagram

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127136 - Nguyễn Thanh Trọng

Edited by:

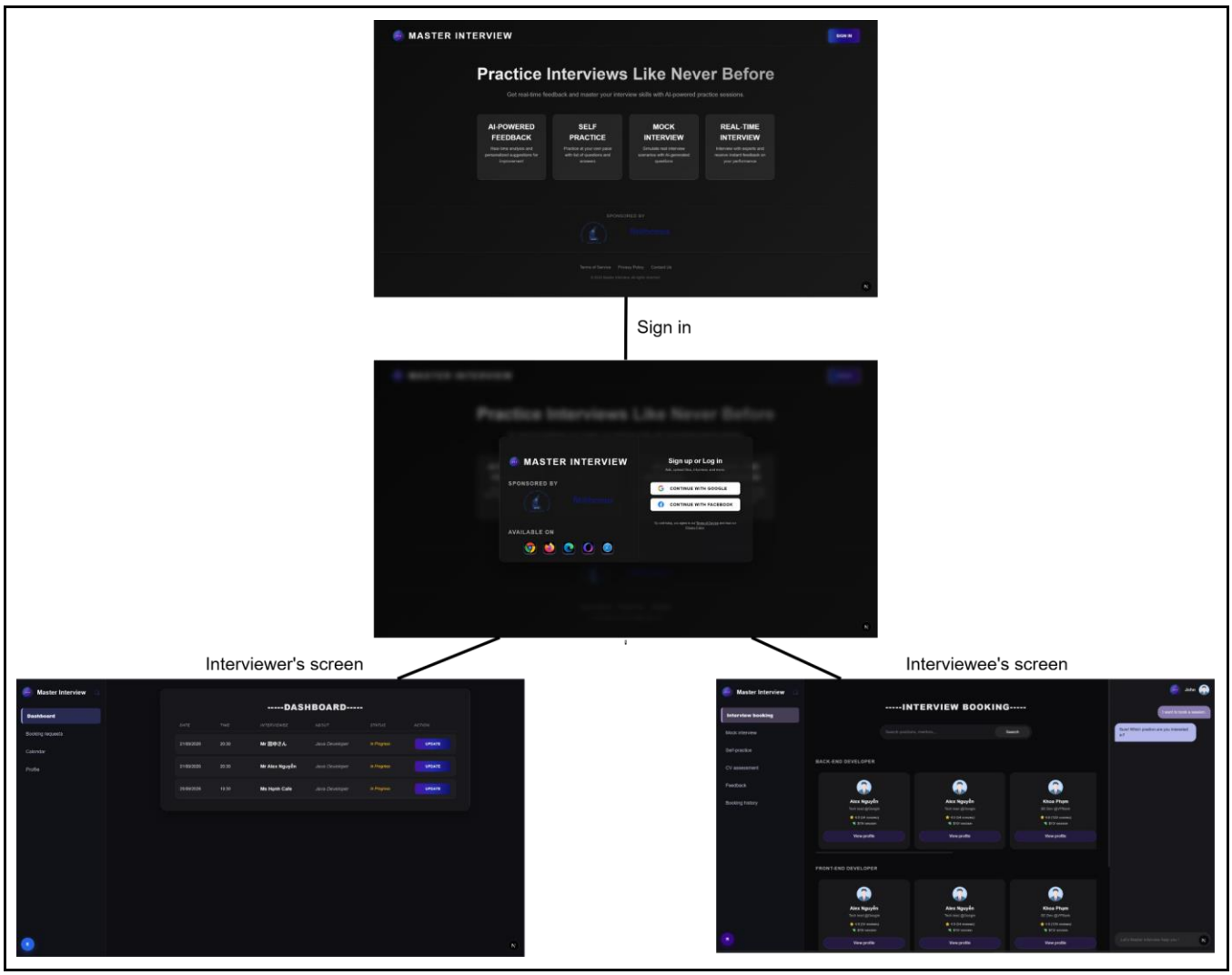
Reviewed by:

[Draw a screen diagram illustrating the relationships and transitions between the screens]

[List the screens]

No	Seq	Screen	Description
1	5.1	Landing	Entry screen where users sign up or log in via Google or GitHub OAuth. Displays the platform branding, sponsor logos, and supported browsers before routing authenticated users into the main dashboard.
2	5.2.1	Interviewee's Booking	Lets interviewees search and browse mentors/interviewers grouped by role (e.g. Back-end, Front-end, Data Engineer), view a mentor's full profile, and book an interview slot by selecting a date, time slot, and uploading a CV, followed by payment confirmation via Stripe.
3	5.2.3	Interviewee's Self Practice	Provides a library of practice questions or exercises the interviewee can work through independently, without a live interviewer, to prepare for common technical or behavioral interview topics.
4	5.2.4	Interviewee's CV Assessment	Lets the interviewee upload their CV and select a target position; the system analyzes it and returns an overall score, a match score against the position, layout feedback, and section-by-section feedback (experience, skills, education, projects).

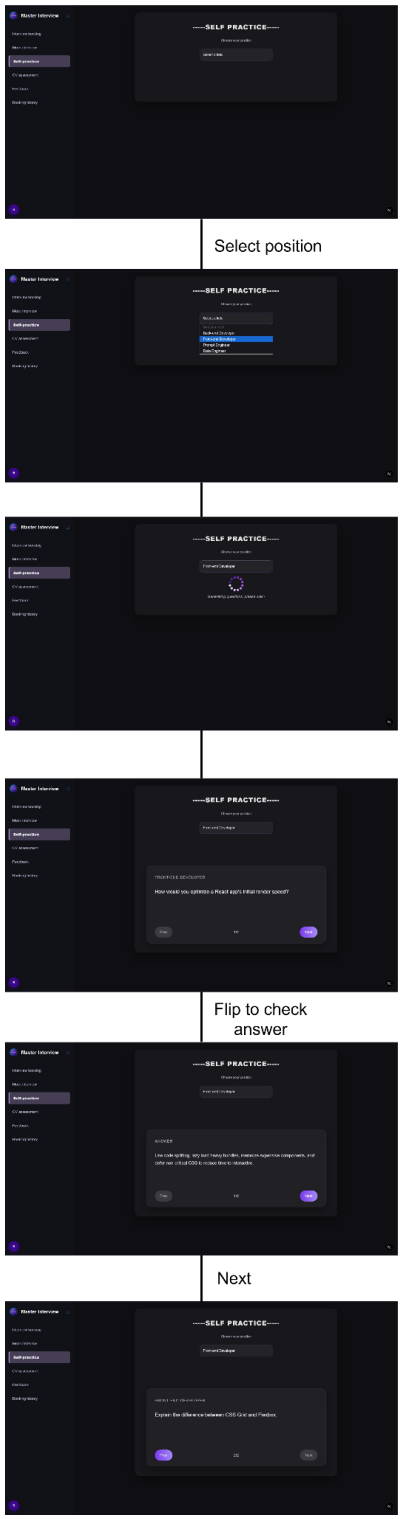
5	5.2.6	Interviewee’s Booking History	Shows a chronological list of the interviewee's past and upcoming bookings, including status (pending, in progress, done, cancelled), interviewer, date/time, and links to meeting details or receipts.
6	5.3.1	Interviewer’s Dashboard	Central overview screen for interviewers showing upcoming sessions, pending booking requests, and summary statistics.
7	5.3.2	Interviewer’s Requests	Lists incoming booking requests from interviewees awaiting the interviewer's confirmation, allowing the interviewer to accept, decline, or reschedule each request.
8	5.3.3	Interviewer’s Calendar	A calendar view where the interviewer manages their availability, blocks out unavailable time slots, and views confirmed sessions across days/weeks/months.
9	5.3.4	Interviewer’s Profile	Lets the interviewer edit their public profile — bio, work experience, languages, hourly/session fee — the same information shown to interviewees in the Booking screen's profile popup.


MASTER INTERVIEW
Practice Interviews Like Never Before
AI-POWERED FEEDBACK
SELF PRACTICE
MOCK INTERVIEW
REAL-TIME INTERVIEW
Sign in
MASTER INTERVIEW
Sign up or Log in
CONTINUE WITH GOOGLE
CONTINUE WITH FACEBOOK
Interviewer's screen
Interviewee's screen
-----DASHBOARD-----
-----INTERVIEW BOOKING-----

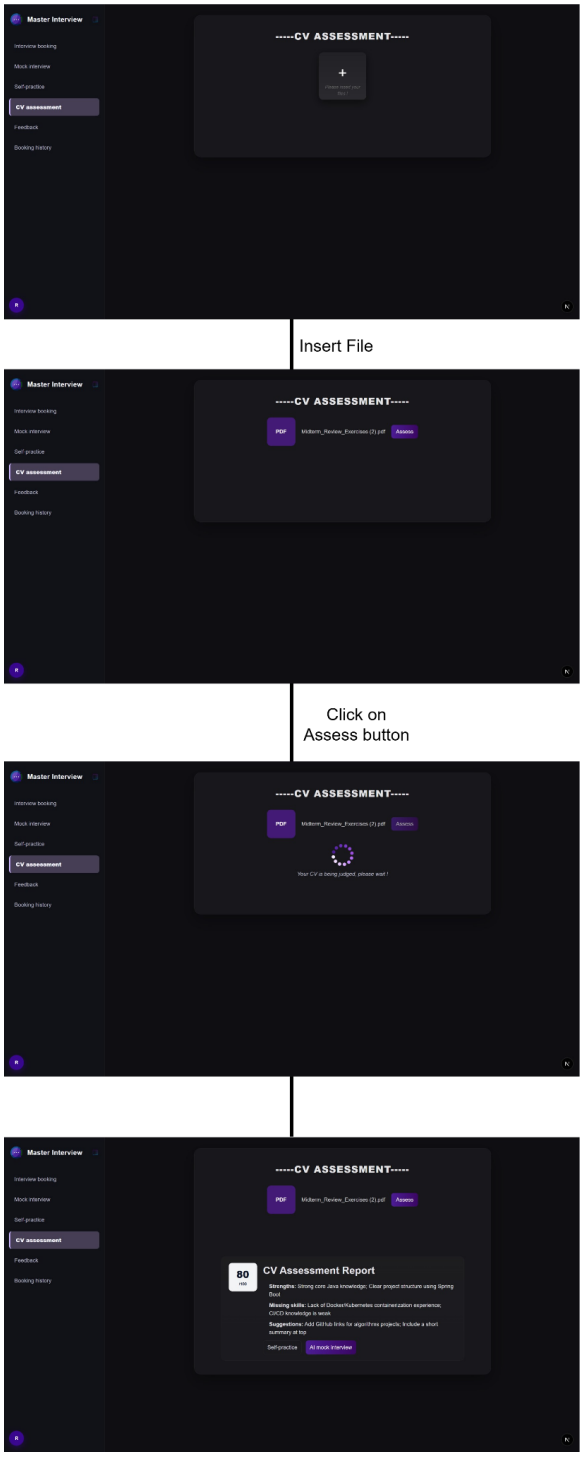
5.1 Landing


-----INTERVIEW BOOKING-----
View profile
Open chatbot on
top-right corner to interact
Book
Pay
After booking,
informations stored in
Booking History
-----BOOKING HISTORY-----

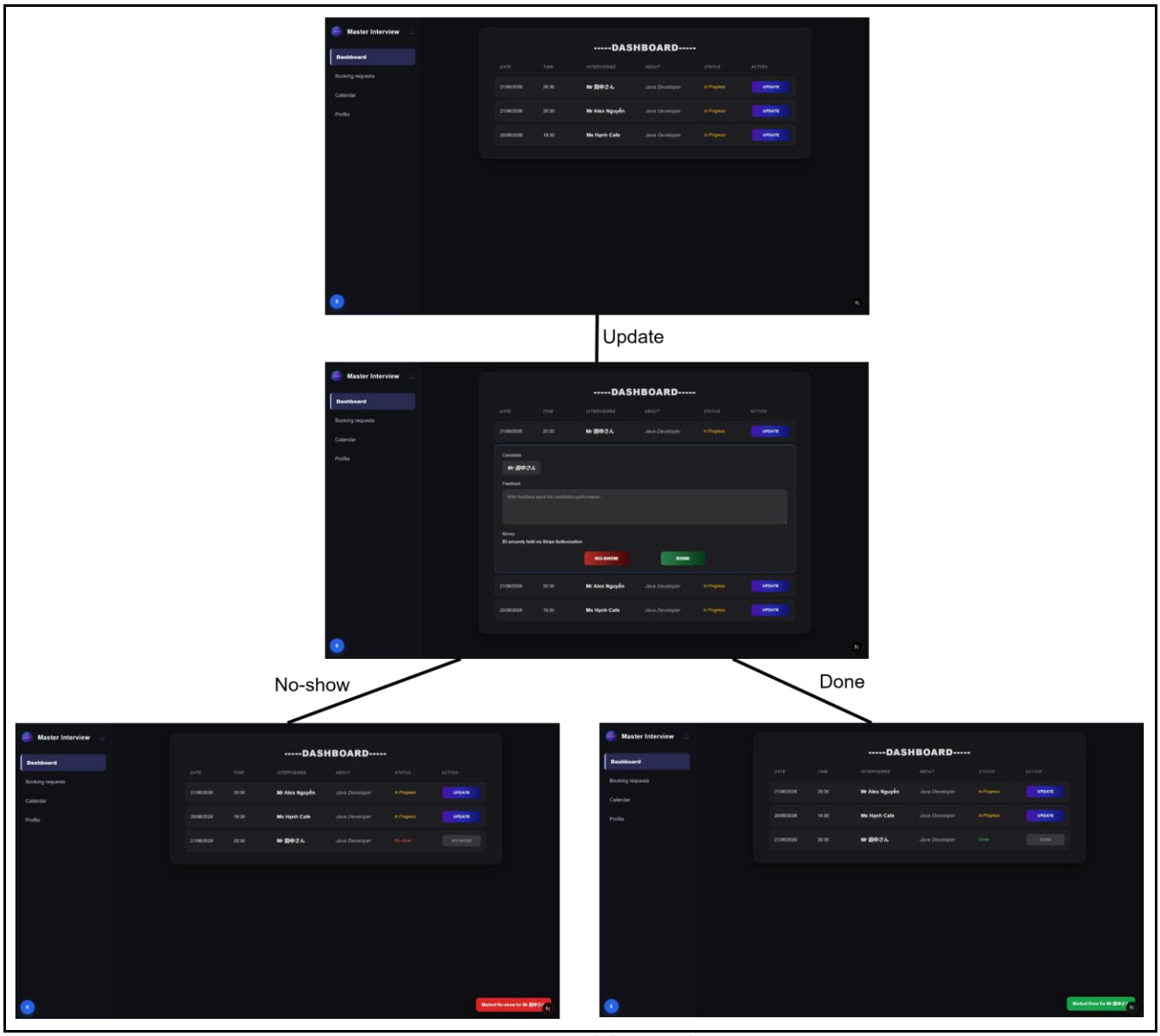
5.2.1 Booking



5.2.3 Self Practice

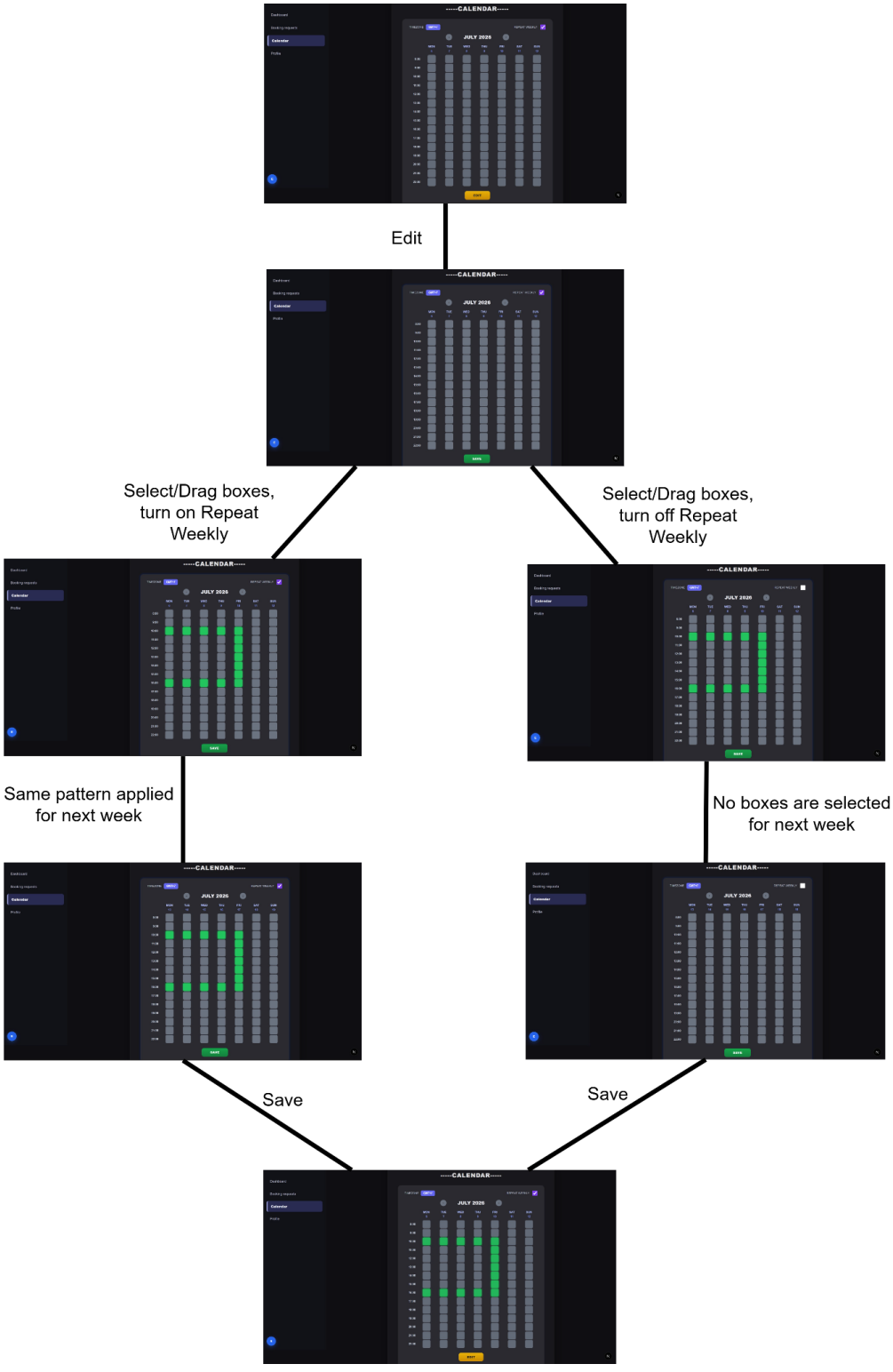


5.2.4 CV Assessment

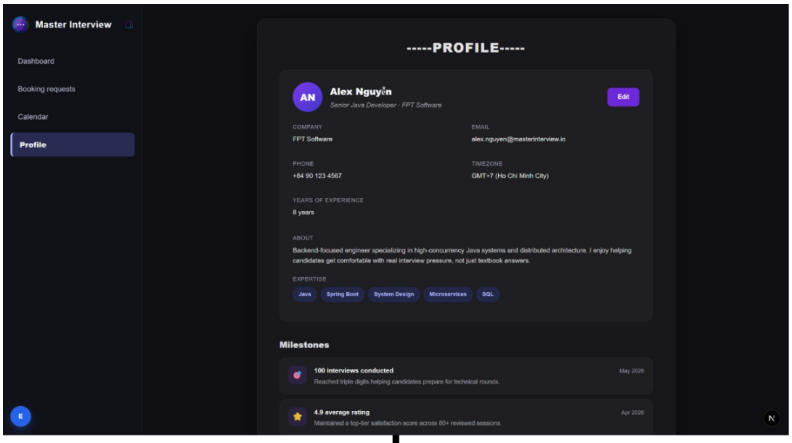


5.3.1 Dashboard

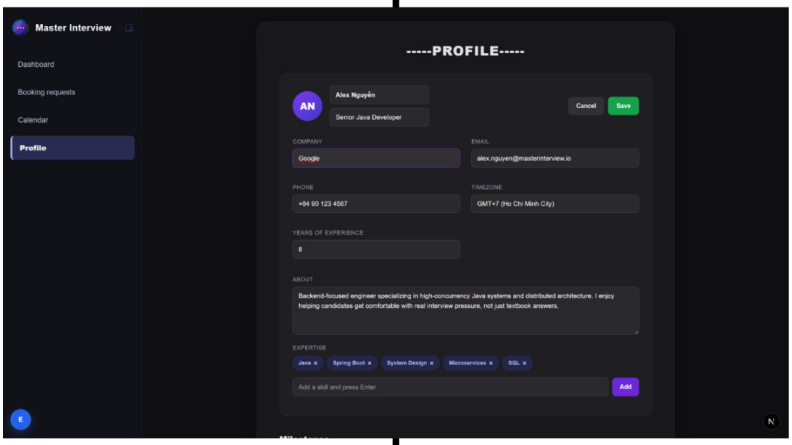
5.3.2 Requests



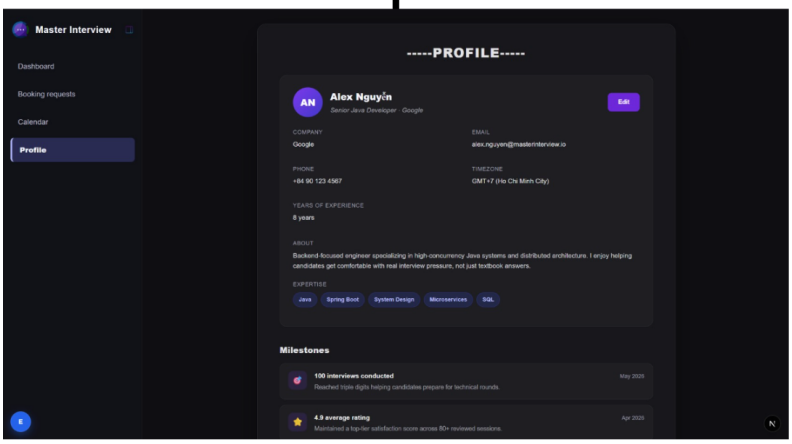
5.3.3 Calendar



Edit



Save



5.3.4 Profile

1.7 Screen Specifications

[Students should select and present the specifications of a few (5-6) of the most important screens. For the remaining screens, only the user interface design needs to be drawn]

1.7.1 Screen "A"

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127136 - Nguyễn Thanh Trọng

Edited by:

Reviewed by:

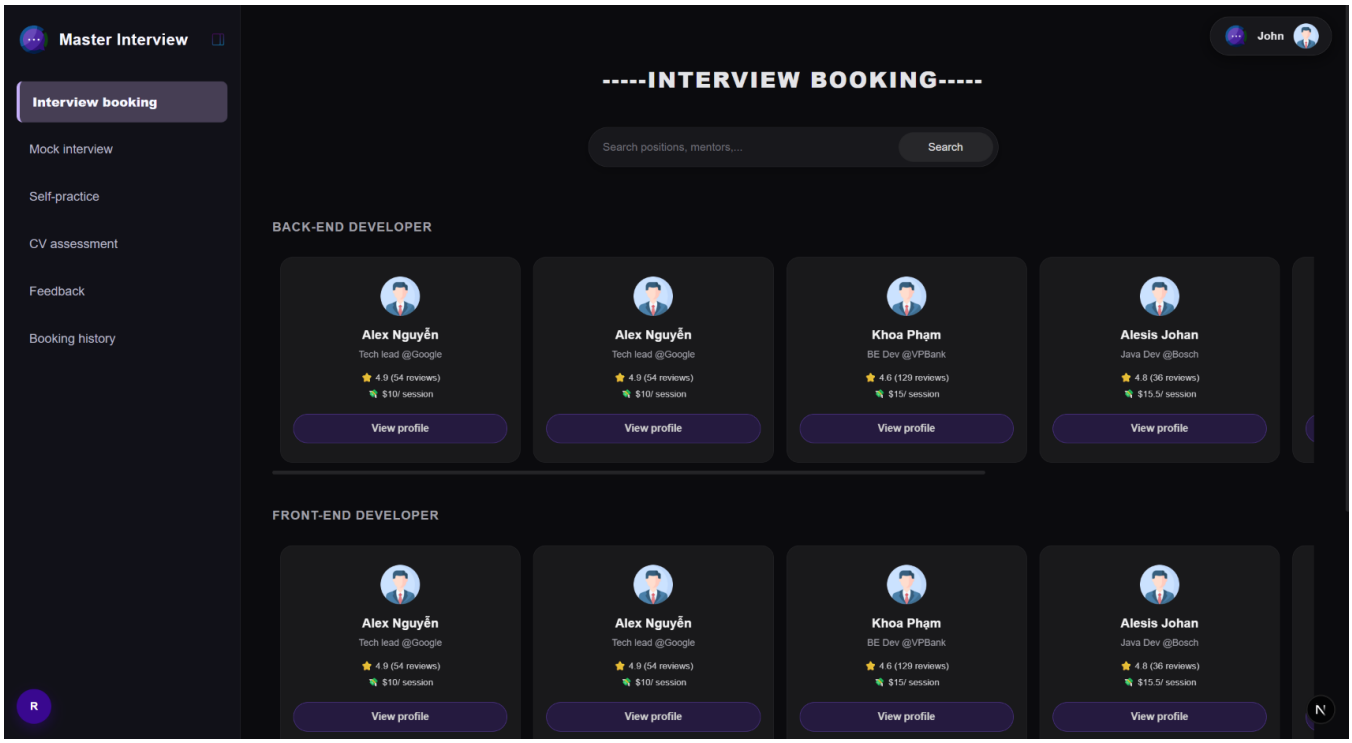
A specific function for interviewee is Booking. This screen uses a three-column layout: a fixed left navigation sidebar, a scrollable center content area, and a collapsible right chat panel.

- **Left sidebar** — displays the "Master Interview" logo/brand, a collapse toggle, and the main navigation list (Interview booking, Mock interview, Self-practice, CV assessment, Feedback, Booking history), with the active item ("Interview booking") highlighted in purple. A user avatar sits fixed at the bottom-left.
- **Center content** — headed by a dashed-style title ("-----INTERVIEW BOOKING-----"), followed by a centered search bar ("Search positions, mentors,..." + Search button), then a series of horizontally-scrollable mentor sections grouped by role (Back-end Developer, Front-end Developer, Data Engineer, ...). Each mentor is shown as a card with avatar, name, role/company, star rating with review count, session fee, and a "View profile" button. The grid dynamically reflows from 5 to 3 columns depending on whether the side panels are expanded or collapsed *(Image 4 vs. Image 1)*.
- **Right chat panel** — an AI assistant panel that can be toggled open/closed; when open it shows a scrollable message thread (user messages right-aligned in purple, assistant replies left-aligned) and a text input pinned to the bottom ("Let's Master Interview help you!").
- **Mentor Profile popup** *(Image 2)* — a modal overlay triggered by "View profile," styled with a gold gradient border, showing the mentor's avatar, name, role, and rating in the header; an "About me" panel (purple gradient) on the left; "Work experience," "Languages," and "Fee" sections on the right; and BOOK / CANCEL actions at the bottom.
- **Booking/Payment popup** *(Image 3)* — a second modal replacing the profile popup after BOOK is clicked, split into two equal columns: a calendar + available time-slot picker on the left, and payment/booking details on the right, with PAY/CONFIRM and CANCEL actions.

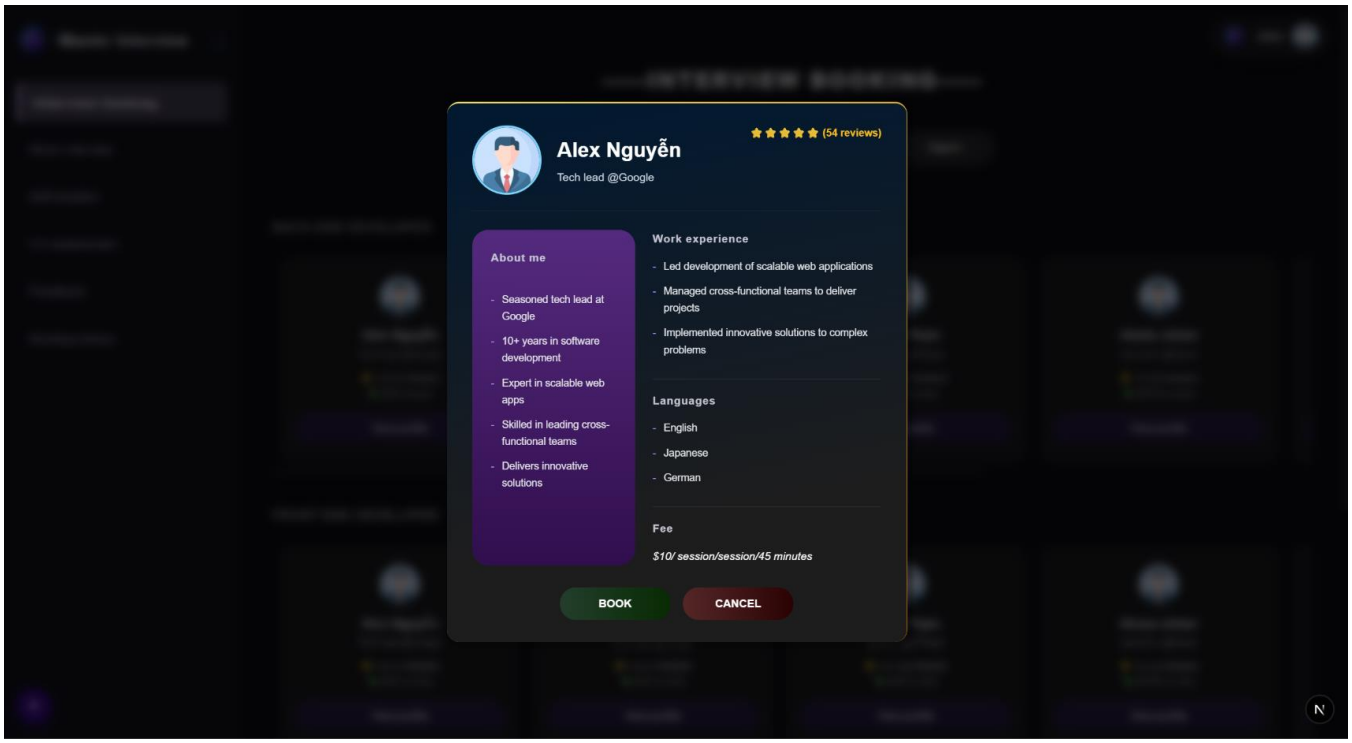
EVENT HANDLING

Event	Trigger	Result
Collapse/expand sidebar	Click toggle icon next to "Master Interview"	Sidebar narrows/widens; mentor card grid reflows column count accordingly
Collapse/expand chat panel	Click chat icon / floating trigger pill	Chat panel slides open or closed; grid reflows to reclaim/give up width
Search input	Type in search field	Live-filters and shows a dropdown of up to 5 matching mentors by name, position, or company
Search button click	Click "Search"	Opens/refreshes the dropdown using current input text
Select search result	Click a dropdown item	Opens that mentor's Profile popup directly
Scroll section	Click "‹" / "›" arrows on a section	Smoothly scrolls that section's card row left/right by a fixed offset
View profile	Click "View profile" on a card	Opens the Mentor Profile popup for that mentor
Close profile popup	Click "CANCEL" or click outside the modal	Closes the popup, clears selected mentor
Book	Click "BOOK" in profile popup	Transitions to the Booking/Payment popup for the same mentor
Change month	Click "‹" / "›" on calendar header	Navigates calendar to previous/next month (previous disabled if it would go before today)
Select date	Click a day cell	Highlights the selected date; disabled for past dates
Select time slot	Click a time-slot pill (e.g. "07:00-08:00")	Highlights the selected slot, deselecting any previous choice
Upload CV	Click the CV dropzone	Opens file picker; selected filename displayed in place of the prompt text
Confirm/Pay	Click "CONFIRM" / "PAY"	Submits booking (and CV) to the backend; on success shows a Stripe payment step, then a success notice; on failure shows an inline error

		message
Cancel booking	Click "CANCEL" in the booking popup	Closes the popup without submitting, returns to the booking list
Send chat message	Type in chat input and press Enter	Appends the message to the thread and scrolls to the latest message

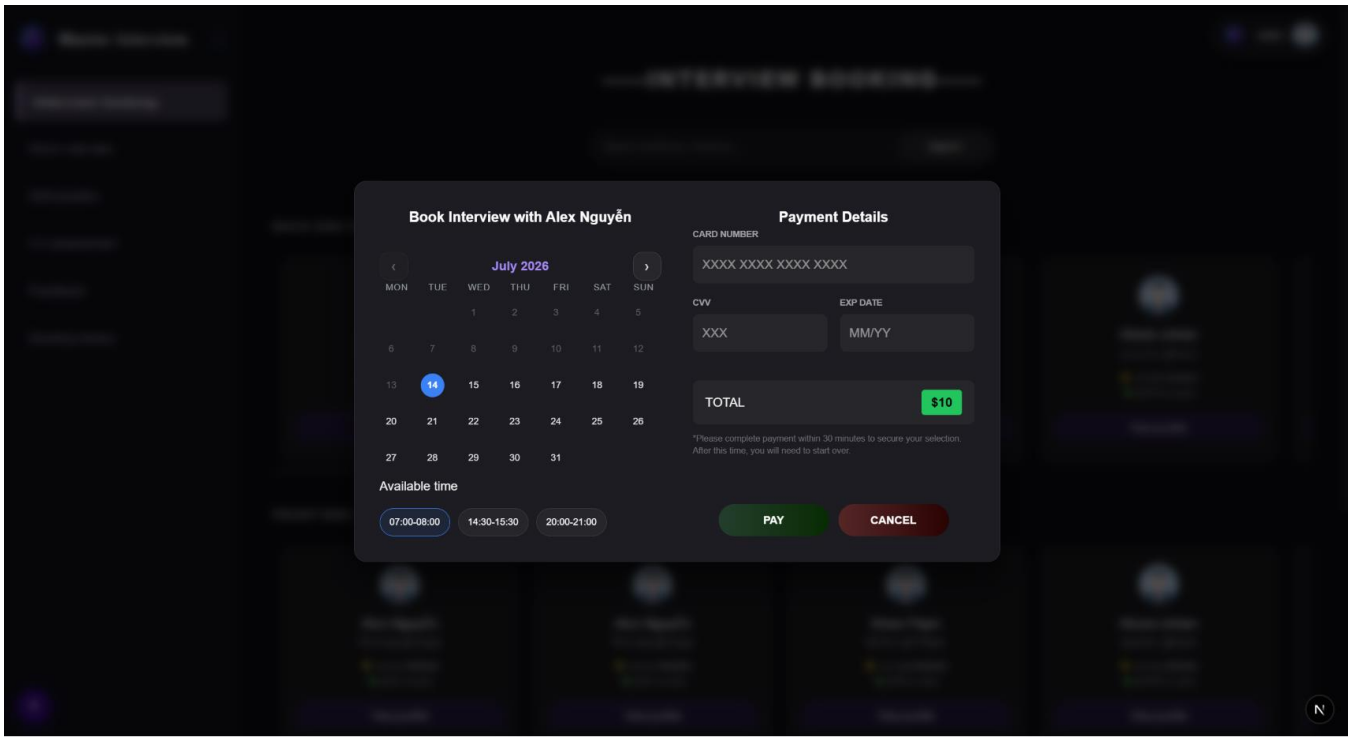


5.2.1.1 Default booking list state



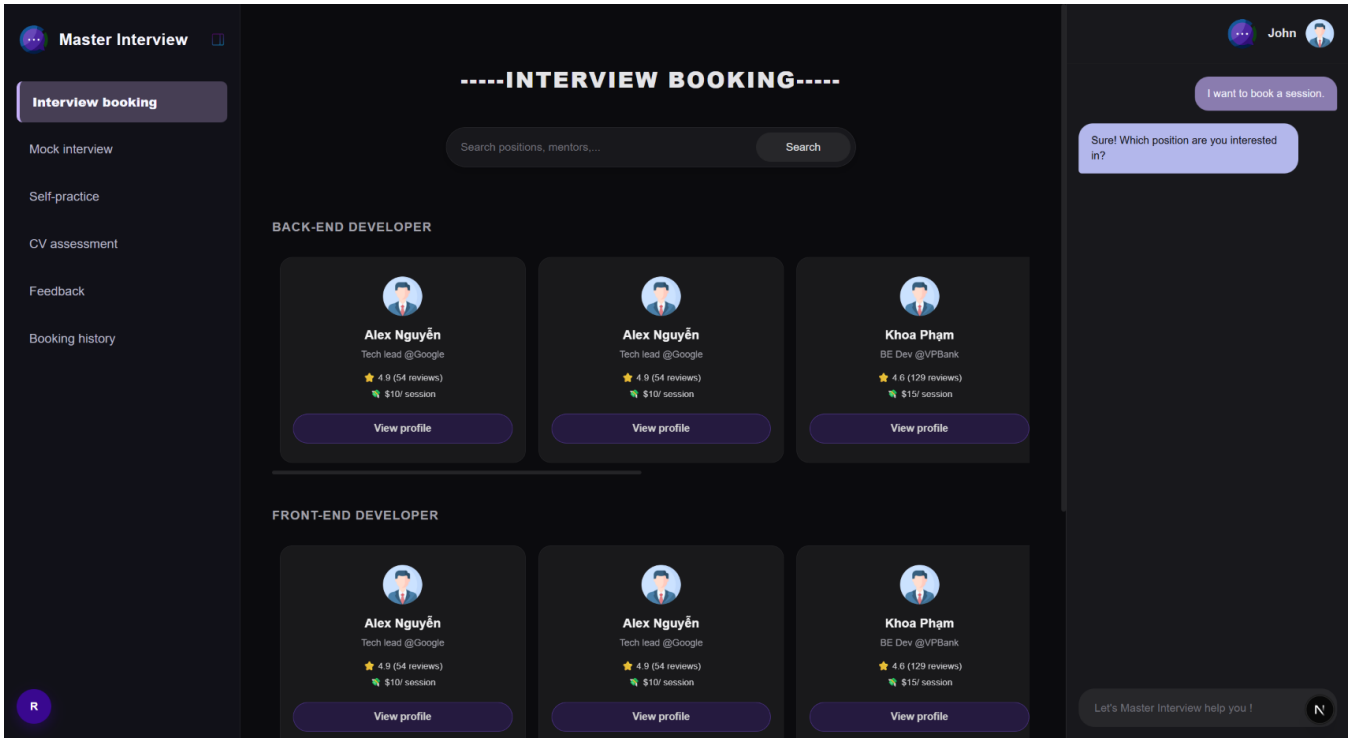
5.2.1.2 Mentor's profile pop-up

Interviewer's information is shown, user can decide whether to book by clicking "Book" or check another interviewer's profile by clicking "Cancel".



5.2.1.3 Booking payment & Date selection pop-up

A small calendar appears on left side to announce available days of interviewer, user can select appropriate date and time to get an appointment. The right sight of pop-up is payment details, user need to input the required information. After aforementioned actions are done, click "Pay" to complete booking, or else, clicking "Cancel" to quit the payment pop-up.



5.2.1.4 Booking with chatbot

A supportive agent is located on the right side. All of the aforementioned actions can be done just by communicating with agent, from search,show profile,... can be shown and done directly in this chatbot.

1.7.2 Screen "B"

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127136 - Nguyễn Thanh Trọng

Edited by:

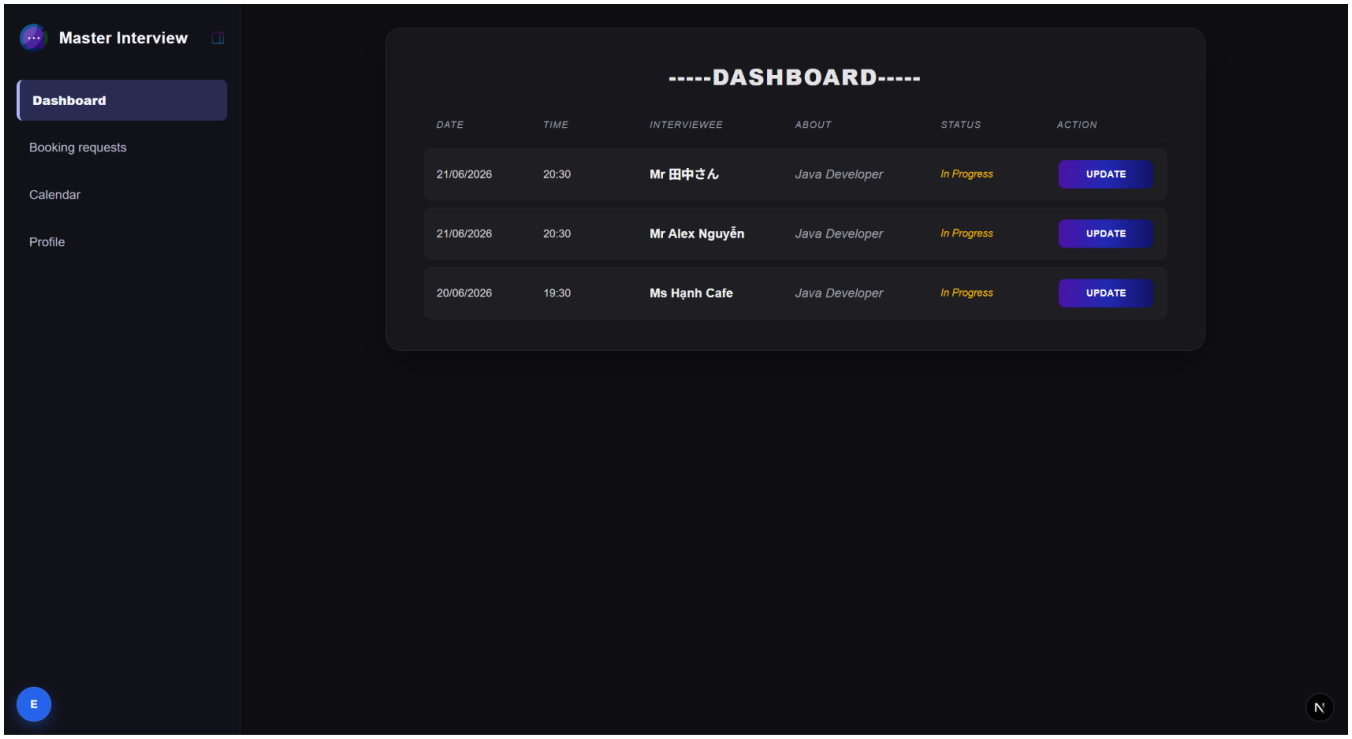
Reviewed by:

A specific function for interviewer is updating and giving feedback about interviewee. This screen uses a two-column layout: a fixed left navigation sidebar, a dashboard in content area.

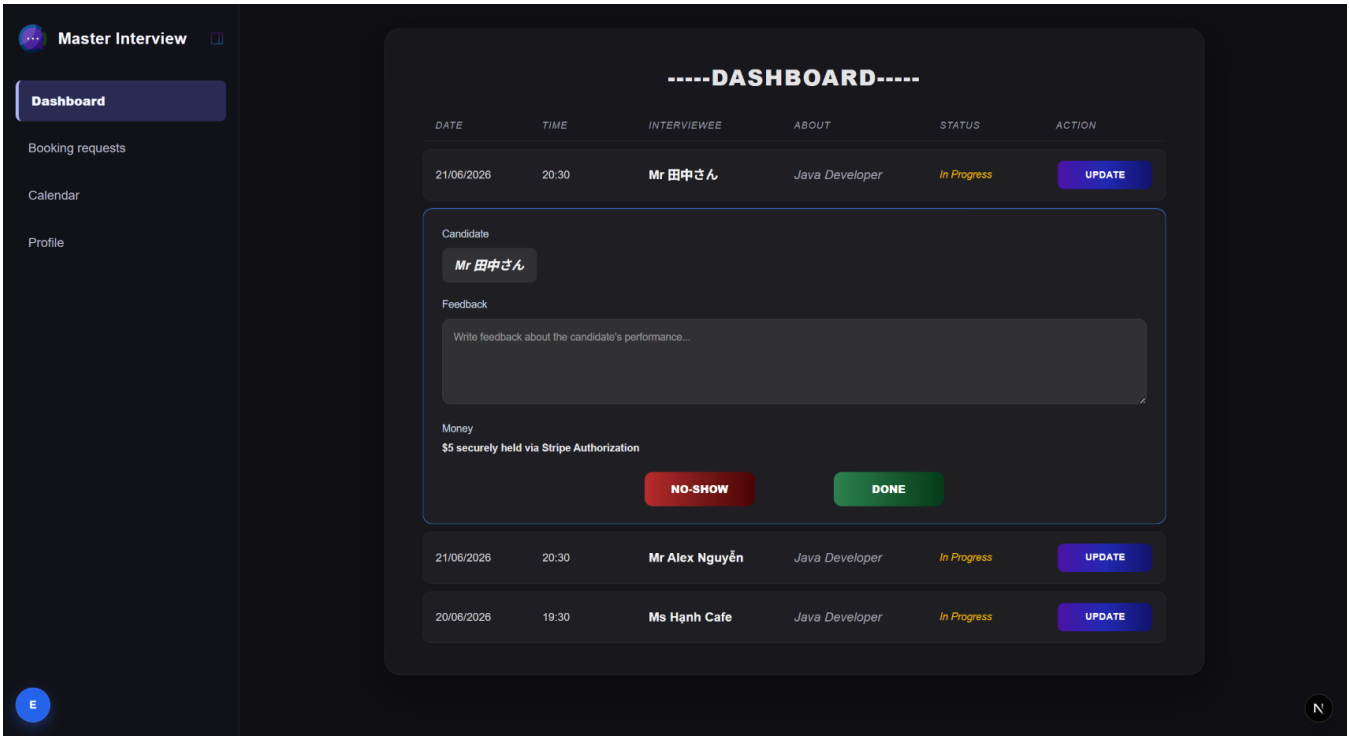
- **Left sidebar** (dark navy/black): App logo "Master Interview" at top, with a collapse toggle icon. Below it, a vertical nav menu: **Dashboard** (highlighted as active), **Booking requests**, **Calendar**, **Profile**. A user avatar circle ("E") sits at the bottom of the sidebar.
- **Main panel**: A single card with rounded corners on a black background, centered under a stylized heading "-----DASHBOARD-----" in a monospace/retro font.
- **Table structure**: Column headers in gray italic caps — DATE, TIME, INTERVIEWEE, ABOUT, STATUS, ACTION — with each row underneath separated by thin dividers.
- **Toast notifications** appear bottom-right in a red bar for confirmation messages.

EVENT HANDLING

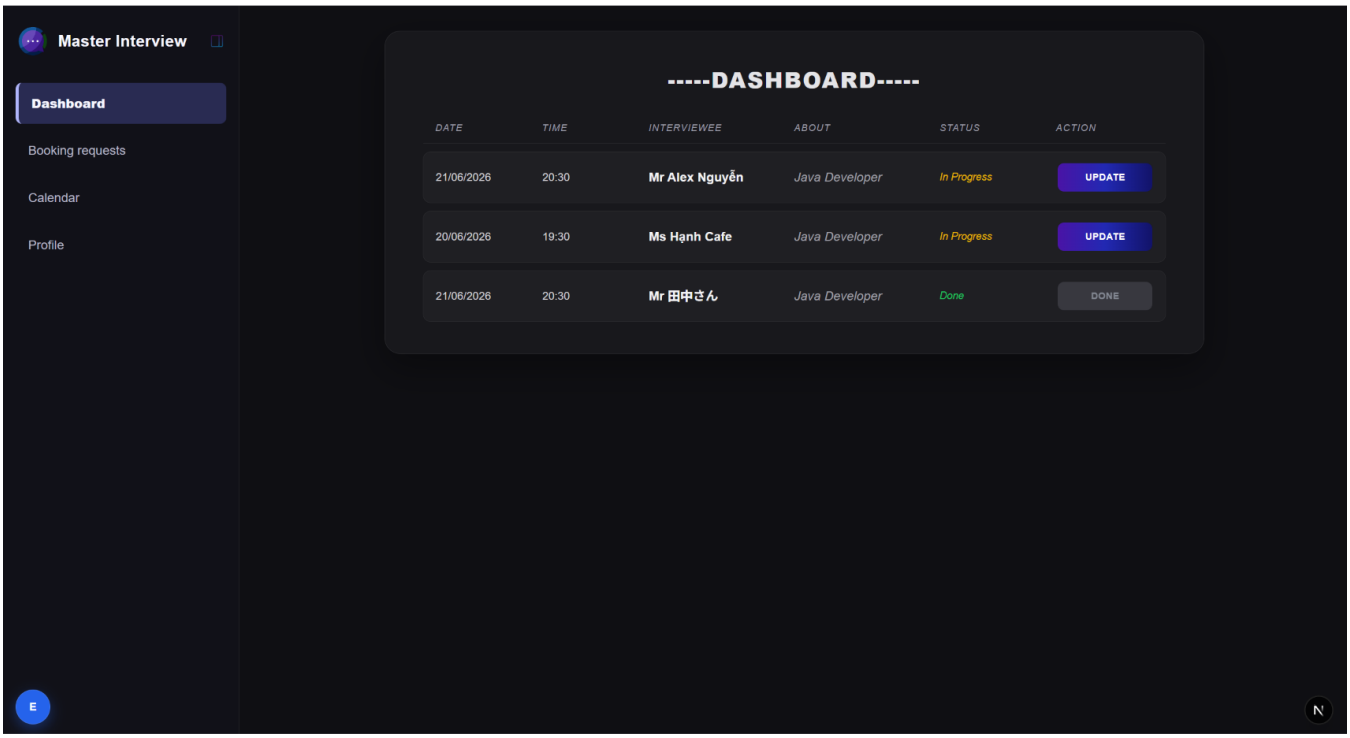
Event	Trigger	Result
View dashboard	Page load	Table lists all interviews sorted by date/time, status badge shown per row
Expand row	Click UPDATE	Inline panel opens with candidate name, feedback field, payment status, and NO-SHOW/DONE controls
Mark no-show	Click NO-SHOW in expanded panel	Row status updates to "No-show," button becomes disabled, row moves to bottom, toast confirms the change, panel collapses
Mark done	Click DONE	Not shown, but presumably captures the $5 payment and marks status "Done" or "Completed"



5.3.1.1 Default dashboard view



5.3.1.2 Feedback box extended



5.3.1.3 Status updated

6 AI Usage Declaration

Based on the provided AI Usage Guideline document, teams must submit their AI usage declaration here. Students are encouraged to leverage AI tools creatively and effectively in this project.

7 Presentation

Teams are required to record presentation videos of their work using the provided template.
Every team member must participate in the presentation, specifically covering the sections their contributed to the project.
Videos must not exceed 30 minutes.
Please upload your videos to Youtube (set to either Unlisted or Public).
Provide the Youtube links the the field below.

https://youtu.be/LowgpfAc6to

8 Reflective Report

1. Most Helpful Sections for Implementation:

- **Class Diagram & Class Specifications:** These sections are the most critical for backend development. They act as a direct blueprint for coding. For example, a clear Class Specification makes it straightforward to generate entities, services, and controllers in Java, as all the methods, parameters, and return types are already strictly defined.
- **Data Diagram & Data Specification:** Essential for database initialization. These documents translate directly into database schemas, foreign key constraints, and ORM mapping configurations (such as Spring Data JPA entities), ensuring data integrity from the start.
- **Screen Diagram & Screen Specifications:** Highly valuable for frontend implementation. They provide exact guidance on building UI layouts and component hierarchies (useful for frameworks like React). The specifications detail input validations, state changes, and the exact triggers for API calls.

2. Least Helpful / Unnecessary Sections for Direct Implementation:

- **Conceptual Model:** While useful during the initial requirements gathering phase to understand the general business domain, it is too abstract for actual coding. During the implementation phase, developers rely entirely on the more detailed **Data Diagram** and **Class Diagram**. Therefore, referring back to the Conceptual Model becomes redundant when writing the actual software.